

A PROJECT REPORT ON

Battery Life Forecasting Using Multi-Model Regression and Deep Feature Fusion

in the
Summer internship programme for the degree of
Bachelor of Technology
in
Computer Science Engineering

Submitted by
ROHIT SAHU (Reg.No:2301301120)

Under the supervision of
Prof (Dr.) RANJAN KUMAR BEHERA
Dr. RANJEET MANDAL



**DEPARTMENT OF ELECTRICAL ENGINEERING
ENGINEERING,
INDIAN INSTITUTE OF TECHNOLOGY, PATNA**



**DEPARTMENT OF COMPUTER SCIENCE ENGINEERING,
VIGNAN INSTITUTE OF TECHNOLOGY AND
MANAGEMENT, BRAHMAPUR**

Date: 09/07/2025

CERTIFICATE OF THE SUPERVISOR(S)

This is to certify that the project entitled "**Battery Life Forecasting Using Multi-Model Regression and Deep Feature Fusion**" being submitted to the Indian Institute of Technology, Patna, by **ROHIT SAHU (Reg. No. 2301301120)**, for the completed an internship program from **08/06/2025** to **09/07/2025**, is carried out by his, in the Department of Electrical Engineering, IIT, Patna, under our supervision and guidance.



Dr. Ranjeet Mandal

Research Fellow

Department of Electrical Engineering,
Indian Institute of Technology, Patna



Prof (Dr.) Ranjan Kumar Behera

Associate Professor

Department of Electrical Engineering,
Indian Institute of Technology, Patna

Acknowledgment

I would like to extend my deepest gratitude towards **Prof (Dr.) Ranjan Kumar Behera** and **Dr. Ranjeet Mandal** to the Department of Electrical Engineering, Indian Institute of Technology, Patna, for his valuable guidance, productive feedback, and endless support during the duration of this study. Their invaluable input and encouragement contributed significantly towards developing this study.

Rohit Sahu
Vignan Institution of Engineering, Berhampur
JULY 9, 2025

Contents

1	Introduction	1
1.1	Introduction	1
1.2	Objective	1
2	Literature review	2
2.1	Literature review	2
2.2	Regression	2
2.3	Types of regression	3
2.3.1	Linear regression	3
2.3.2	Decision tree	4
2.3.3	KNN algorithm	5
2.3.4	Gradient Boosting Regressor	6
2.3.5	eXtreme gradient boosting regressor	7
2.3.6	Light gradient boosting machine regressor	7
2.4	Metrics to evaluate machine learning algorithms	8
2.4.1	Mean absolute error	8
2.4.2	Mean squared error and Root mean squared error	8
2.4.3	R^2 metric	8
2.5	SHapley Additive exPlanations	9
2.6	Deep Learning and Embeddings	9
2.6.1	Deep Learning and Neural Networks	9
2.6.2	Embeddings	10
3	Methodology	11
3.1	Dataset Details	11
3.2	Data Exploration	11
3.2.1	Batch-1 Analysis	12
3.2.2	Batch-2 Analysis	16
3.2.3	Batch 3 Analysis	18
3.2.4	Merged Batch Dataset	19
3.2.5	Data Cleaning, Cycle Selection, and Imputation	20
3.2.6	Feature Aggregation and Quantile-Transformed Regression Pipeline	21
3.2.7	Cycle Life Prediction Using Quantile-Transformed XGBoost with Calibration	21
3.2.8	Cycle Life Prediction Using LightGBM with Quantile Normalization and Calibration	22
3.2.9	Cycle Life Prediction Using Decision Tree Regression with Calibration	23
3.2.10	Cycle Life Prediction Using K-Nearest Neighbors Regression	25
3.2.11	Visualizing Feature Impact on Battery Life Prediction with SHAP	27
3.2.12	Cycle Life Prediction Using Deep Neural Network	28
3.2.13	K-Nearest Neighbors Regression on Deep Feature Embeddings	29
3.2.14	XGBoost Regression Using Deep Features	30
3.2.15	LightGBM Regression with Combined Raw and Deep Embedding Features	31
3.2.16	Teacher-Student Framework for 10-Cycle Early Prediction using LGBM Regressor	32
3.2.17	Multi-Student Knowledge Distillation Using KNN+DL Teacher	33
3.2.18	Teacher Selection Justification	33
3.2.19	Teacher-Student Framework for 10-Cycle Early Prediction using KNN and DL	34
4	Comparison	35
5	Observations	38
6	Conclusion	39
7	Future Scope	39

Abstract

The accurate estimation of lithium-ion battery cycle life remains a critical challenge in the development and deployment of advanced energy storage systems, particularly in applications such as electric vehicles and grid-scale storage. This study proposes a comprehensive data-driven framework for early prediction of battery lifespan based on multi-model regression and deep feature fusion. Utilizing a publicly available dataset comprising 124 commercial lithium iron phosphate (LFP)/graphite cells, we conduct extensive exploratory data analysis and statistical feature extraction from the initial 50 charge-discharge cycles. A suite of machine learning algorithms—including Linear Regression, Decision Tree Regression, K-Nearest Neighbors (KNN), Light Gradient Boosting Machine (LightGBM), and Extreme Gradient Boosting (XGBoost)—is employed, augmented with advanced techniques such as quantile transformation, missing data imputation, and isotonic regression-based calibration to enhance predictive accuracy.

To improve model generalization and interpretation, feature importance is assessed using Shapley additive explanations, revealing the dominant influence of parameters such as internal resistance and discharge capacity on cycle life. Furthermore, deep neural networks (DNNs) are utilized to extract latent feature embeddings, which are subsequently integrated with handcrafted statistical features to form a hybrid feature representation.

Additionally, a teacher-student distillation framework is implemented for early-life prediction using only the first 10 cycles of data. Among various configurations, a LightGBM-based student model trained on pseudo-labels from a LightGBM teacher demonstrates the best performance and underscoring the efficacy of early-cycle data in capturing degradation signatures. The proposed multi-model framework offers a scalable, interpretable, and accurate solution for lithium-ion battery life prediction, facilitating accelerated screening and informed decision-making in battery management systems.

Chapter-I

Introduction

1 Introduction

1.1 Introduction

With the increasing demand for clean energy technologies and electrification of transportation lithium-ion batteries (LIBs) have established themselves as the leading energy storage solution across a wide range of application including, electrical vehicles (EVs), smart grids, and portable electronic devices. Their widespread use is driven by favourable characteristics such as high energy density, long service life, and stable cycling performance [1, 2]. However, for accurately estimating the cycle life of LIBs remains a fundamental challenge due to the complex and non-linear nature of degradation mechanisms. Traditional lifetime assessment approaches often require full-cycle ageing tests under controlled condition until the battery drops to 80% of its original capacity which is both time-consuming and resource-intensive [2, 3]. This creates a barrier in battery development, production, and deployment especially as applications scale in size and scope. There is thus a strong motivation to identify faster, cost-effective, and reliable alternatives for assessing battery longevity early in its life-cycle.

One emerging approach is early-cycle life prediction, which seeks for forecasting a battery's total lifespan using data collected from the initial portion of its usage typically the first 5 to 10% for its total cycles [1]. This early phase, although characterized by minimal capacity fade, contains critical information about internal transformations within the battery including the formation and stabilization of the solid electrolyte interphase (SEI) layer, lithium plating, and mechanical stress-induced microstructural changes in electrode materials [1, 3]. These changes affect measurable parameters such as voltage profiles, internal resistance and charge/discharge behavior, which can be ranked as input features for predictive modeling. The benefits of early predictions are considerable, ranging from accelerating cell screening in manufacturing to identifying potential defects before batteries are deployed in the field. Furthermore, early-stage insights can reduce time and cost during battery development, improve system reliability, and support predictive maintenance approach in real-world applications [2, 4].

To analyze early-cycle behavior and derive reliable lifetime predictions, researchers are increasingly turning to data-driven approaches, particularly machine learning (ML). ML models are well suited for capturing complex knowledge linear dependencies in large datasets without requiring detailed mechanistic knowledge for battery degradation [5]. These models can ingest early cycle data including voltage, current, and temperature profiles which extract hidden patterns that correlates with long-term aging behaviors [4]. Hybrid architectures, such as conventional neural networks (CNNs) and attention based frameworks have been proposed to improve prediction performed by identifying relationships between features and degradation trends [4, 3]. Compared to physics based or semi empirical models, ML approaches offer scalability and adaptability across different battery chemistries, operating conditions, and usage pattern [5]. The integration of early stage data analysis and ML modeling has thus emerged as a promising avenue to enable faster, more accurate and more practical solutions for battery forecasting in both research and industry.

1.2 Objective

- To develop a machine learning-based framework for accurately predicting the cycle life of lithium-ion batteries using data from the initial 10–50 charge–discharge cycles.
- To evaluate and compare traditional and deep learning models—integrating statistical features, deep embeddings, and SHAP explanations—for scalable, interpretable, and early-life battery health forecasting.

Chapter-II

Review and Regression

2 Literature review

2.1 Literature review

Yang *et al.*(2024) provided a comprehensive review of early-cycle lithium-ion battery life prediction methods, highlighting that combining data-driven, mechanism-based, and hybrid approaches significantly enhances early-stage accuracy while reducing cost and testing time, with emerging models achieving high precision using less than 10% of cycle data[1]. Severson *et al.*(2019) developed a data-driven machine learning model that accurately predicts lithium-ion battery cycle life using only the first 100 cycles—with negligible capacity degradation—achieving a 9.1% test error and enabling early classification of long vs. short life cells with up to 95.1% accuracy[2]. Su *et al.*(2021) demonstrated that a convolutional neural network trained on early-cycle data significantly improves lithium-ion battery cycle life prediction, achieving a test error of just 22.73%, outperforming traditional models based on expert-extracted features[3]. Yang(2021) proposed a hybrid convolutional neural network incorporating charge voltage, current, and temperature data with feature and cycle attention mechanisms, achieving highly accurate lithium-ion battery life and RUL predictions with test errors as low as 1.1% and 3.6%, respectively[4]. Zhang *et al.*(2022) presented a comprehensive review highlighting how various machine learning techniques—especially deep learning, ensemble models, and hybrid architectures—have significantly advanced the prediction of lithium-ion battery performance, health, and thermal management, with reported errors often below 2% for key estimations like state of charge and remaining useful life[5]. Attia *et al.* developed advanced modeling approaches for lithium-ion battery aging and optimization: in 2020, they introduced a closed-loop optimization (CLO) system combining early-prediction models with Bayesian optimization to rapidly identify optimal fast-charging protocols using early-cycle data; while in 2008, they employed a physics-based single particle model to uncover a three-stage capacity fade mechanism involving SEI formation, cathode degradation, and a limiting electrode shift, offering mechanistic insights into long-term performance decline[6, 7]. Yang *et al.* (2017) developed a physics-based lithium-ion battery model that captures the transition from linear to nonlinear aging due to lithium plating, revealing a positive feedback loop between anode porosity loss and lithium deposition rate that leads to rapid capacity fade and resistance rise in aged cells[8].

2.2 Regression

Regression is a statistical and machine learning technique used to model the relationship between a dependent variable and one or more independent variables. It is one of the most fundamental tools for understanding, predicting, and quantifying the influence of input variables on an outcome. The main goal of regression is to estimate the functional relationship that best describes how the target variable changes as a function of the input features.

The general mathematical formulation of a regression model is expressed as:

$$Y = f(X) + \varepsilon \quad (1)$$

where Y is the dependent (or response) variable, X is the set of independent (or explanatory) variables, $f(X)$ is the true but unknown function mapping inputs to output, and ε is the error term accounting for random variability or noise in the data. The objective is to approximate the function f as accurately as possible using the available data.

When the relationship is assumed to be linear, the model simplifies to the linear regression form. For multiple independent variables, the model is written as:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_n X_n + \varepsilon \quad (2)$$

Here, β_0 denotes the intercept, $\beta_1, \beta_2, \dots, \beta_n$ are the coefficients of the corresponding independent variables $X_1, X_2, \dots, X_n$, and ε represents the residual error, i.e., the difference between observed and predicted values.

Regression can be used for both prediction and inference. Predictive regression focuses on estimating the value of Y for new or unseen values of X , whereas inferential regression seeks to understand the nature and strength of the relationship between variables. In particular, inference involves assessing the statistical significance and magnitude of the coefficients β_i . This versatility makes regression suitable for a wide range of fields, including economics, medicine, engineering, and social sciences.

Several assumptions underlie a valid linear regression model. These include:

- **Linearity:** The relationship between the predictors and the outcome is linear.
- **Independence:** Observations and errors are independent of one another.
- **Homoscedasticity:** The variance of residuals is constant across all values of the independent variables.
- **Normality:** Residuals are normally distributed.

Violations of key regression assumptions, such as linearity, independence, and homoskedasticity, can result in biased, inconsistent, or inefficient estimates, thus undermining the model's validity.

Categorical variables are incorporated into regression analysis through techniques like label encoding or one-hot encoding, enabling the model to handle diverse data types. In addition, steps such as feature selection, transformation, and diagnostic checking are often essential to ensure model robustness and interpretability.

Advancements in machine learning have extended regression beyond linear models. Methods such as decision trees, random forests, support vector regression, and neural networks effectively capture complex, non-linear patterns without requiring explicit functional forms. Nonetheless, the core objective of regression—modeling the relationship between inputs and a continuous output—remains central to both traditional statistics and modern machine learning.

In conclusion, regression analysis is a powerful and widely used method for explaining and predicting quantitative relationships. Its combination of interpretability, theoretical rigor, and adaptability makes it an indispensable tool in data science and applied research [9].

2.3 Types of regression

Regression analysis includes a variety of techniques designed to model the relationship between a dependent variable and one or more independent variables. Each type of regression is suited to different kinds of data, modeling assumptions, and analytical goals. Below are the major types of regression methods commonly used in both statistical analysis and machine learning [10].

2.3.1 Linear regression

Linear regression is one of the most commonly used techniques for modeling the relationship between a dependent variable and one or more independent variables. It assumes a linear relationship and aims to fit a straight line that best represents the data.

In simple linear regression, with only one independent variable, the relationship is described by the equation:

$$Y = \beta_0 + \beta_1 X + \varepsilon \tag{3}$$

Where:

- Y is the dependent variable,
- X is the independent variable,
- β_0 is the intercept (the value of Y when $X = 0$),
- β_1 is the slope (the rate of change in Y for a unit change in X),
- ε is the random error term accounting for the variation not explained by the model.

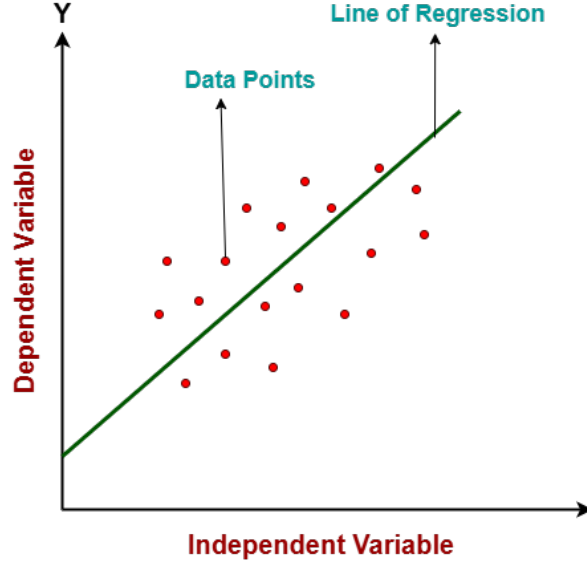


Figure 1: Linear regression

The above Figure 1 provides a visual understanding of how simple linear regression works. In this figure, the horizontal axis (X-axis) represents the independent variable, while the vertical axis (Y-axis) represents the dependent variable. The blue dots indicate the observed data points for various values of the independent variable. The red line is the regression line, representing the predicted relationship between the variables. This line follows the equation.

$$Y = \beta_0 + \beta_1 x \quad (4)$$

where β_0 is the intercept and $\beta_1 x$ is the slope of the line.

The intercept, denoted as β_0 , shows the predicted value of Y when the independent variable X is equal to zero. The slope, β_1 , indicates the rate at which the dependent variable Y changes for a one-unit increase in X . At any specific point X_i , the model predicts a value $\hat{y}_i$ that lies on the regression line. However, the actual observed value y_i may differ from this prediction. The difference between the observed value and the predicted value is called the residual or error term, denoted by ε_i . These residuals are shown as vertical dashed lines in the diagram.

The purpose of linear regression is to find the best-fitting line that minimizes the sum of the squared residuals across all data points. This method ensures that the line is as close as possible to the observed values, making the model both accurate and reliable for prediction. The diagram effectively illustrates these concepts by showing the regression line, the observed and predicted values for a given point, and the error involved in prediction.

2.3.2 Decision tree

A Decision Tree is a tree-structured, nonparametric predictive modeling tool used in supervised machine learning to make decisions or predictions based on input features. It recursively partitions the dataset into subsets based on feature values, with the aim of creating homogeneous groups with respect to the target variable. This model mirrors human decision-making processes and is widely appreciated for its transparency, interpretability, and versatility in classification and regression tasks.

Formally, a decision tree consists of a root node, internal decision nodes, branches, and leaf nodes:

- Each node represents a test or decision based on an attribute.
- Each branch corresponds to the outcome of the test (e.g., "Yes" or "No").
- Each leaf node holds a class label (in classification) or a numeric value (in regression).

The model is trained by learning the the splitting rules that best separate the input data according to a chosen criterion, such as information gain, gain ratio, Gini index, or divergence [11].

A decision tree is composed of different types of nodes, each with a specific role in the decision-making process which is shown in Figure 2.

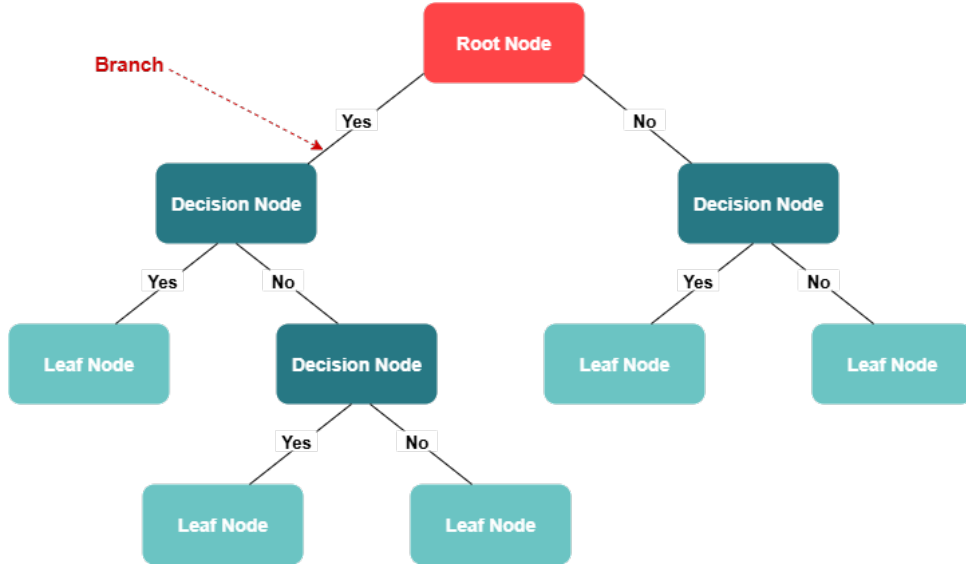


Figure 2: Decision Tree

- **Root Node:**

The root node is the topmost node in the tree and represents the entire dataset. It selects the best attribute to split the data initially, based on a defined metric. This node marks the beginning of the decision process.

- **Internal/Decision Nodes:**

These are non-terminal nodes that perform tests on input features. Each internal node represents a decision rule that partitions the dataset further.

- **Branches (Edges):**

A branch emerges from a decision node and represents the outcome of the node's test. Each branch leads to another decision node or a terminal node. For binary decisions, there are typically two branches (e.g., "True" or "False"), while multi-way splits can produce multiple branches based on categorical values.

- **Leaf Nodes (Terminal Nodes):**

Leaf nodes are the endpoints of the tree. They do not split further and instead assign a class label (for classification problems) or output a numerical prediction (for regression). These nodes encapsulate the final decision made for a given input path through the tree [12].

2.3.3 KNN algorithm

The K-Nearest Neighbor (KNN) algorithm is a fundamental and intuitive method in supervised machine learning, primarily used for classification and regression problems. It belongs to the family of non-parametric, instance-based learning algorithms, meaning it does not require an explicit model training phase. Instead, the algorithm classifies or predicts outputs for new data points based on the similarity (or proximity) to stored examples in the training dataset. Its effectiveness across domains such as image recognition, medical diagnostics, and text categorization stems from its simplicity, adaptability, and relatively high accuracy for well-structured datasets [13]. For a given query point x , the algorithm calculates the distance between x and every instance x_i in the training dataset. The most commonly used distance metric is the **Euclidean distance**, defined as:

$$d(x, x_i) = \sqrt{\sum_{j=1}^n (x_j - x_{ij})^2} \quad (5)$$

where:

- x is the input vector,

- x_i is the i -th training vector,
- n is the number of features (dimensions).

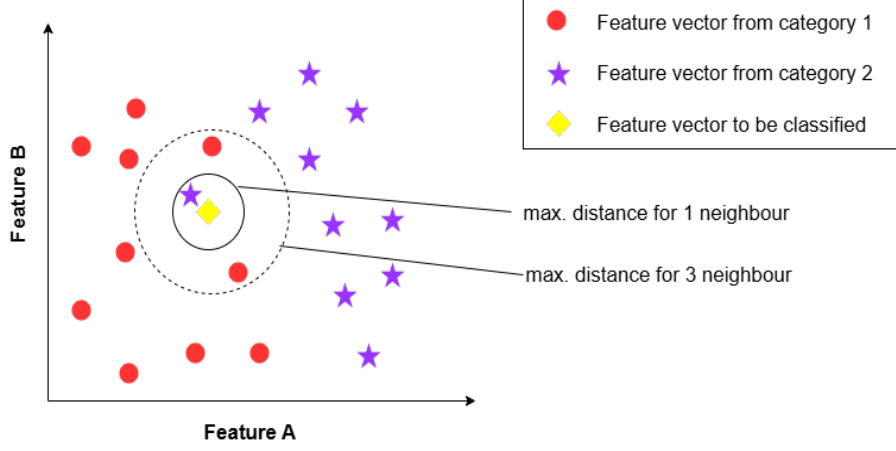


Figure 3: K-nearest neighbor graph

2.3.4 Gradient Boosting Regressor

Gradient boosting is a robust ensemble learning technique that builds a strong predictive model by iteratively combining multiple weak learners—typically regression trees—in a stage wise manner. Letting

$$F_0(x) = \arg \min_{\gamma} \sum_{i=1}^n L(y_i, \gamma) \quad (6)$$

denote the initial model (often the mean of the targets in least-squares regression), gradient boosting progresses through M iterations by computing pseudo-residuals:

$$r_{im} = - \left[\frac{\partial L(y_i, F_{m-1}(x_i))}{\partial F(x_i)} \right] \quad (7)$$

for each training instance i [14]. At iteration m , a base learner $h_m(x)$ (e.g., a shallow decision tree) is fit to these residuals, and its weight γ_m is determined by solving

$$\gamma_m = \arg \min_{\gamma} \sum_{i=1}^n L(y_i, F_{m-1}(x_i) + \gamma h_m(x_i)) \quad (8)$$

The model update incorporates a shrinkage (learning rate) parameter $\nu \in (0, 1]$, yielding

$$F_m(x) = F_{m-1}(x) + \nu \gamma_m h_m(x) \quad (9)$$

which improves generalization by mitigating overfitting [?]. In the special case of squared-error loss $L(y, F(x)) = \frac{1}{n} \sum_i (y_i - F(x_i))^2$, the negative gradient simplifies to the residuals themselves,

$$r_{im} = y_i - F_{m-1}(x_i) \quad (10)$$

and the method reduces to iteratively fitting trees to the errors. This algorithmic structure elegantly frames boosting as a functional gradient descent in function space, where each subsequent weak learner attempts to correct the deficiencies of the ensemble thus far. The final predictor is

$$\hat{F}(x) = F_0(x) + \sum_{m=1}^M \nu \gamma_m h_m(x) \quad (11)$$

achieving high predictive accuracy by sequentially refining the model along directions of steepest descent in the loss landscape. Proper choice of parameters M , tree complexity, and ν yields a well-regularized estimator with strong generalization properties[15].

2.3.5 eXtreme gradient boosting regressor

Extreme Gradient Boosting Regressor is the regression implementation of Extreme Gradient Boosting (XGBoost), a powerful and scalable ensemble learning method based on decision trees. It is particularly well-suited for structured/tabular data and has gained popularity in many real-world regression tasks due to its high performance and flexibility.

How it works:

Figure 4 shows how XGBRegressor builds an ensemble of decision trees, where each new tree attempts to correct the errors made by the previous ones. This approach is called boosting. More specifically, it uses a technique called gradient boosting, where the model is trained in stages using the gradient of a specified loss function[16].

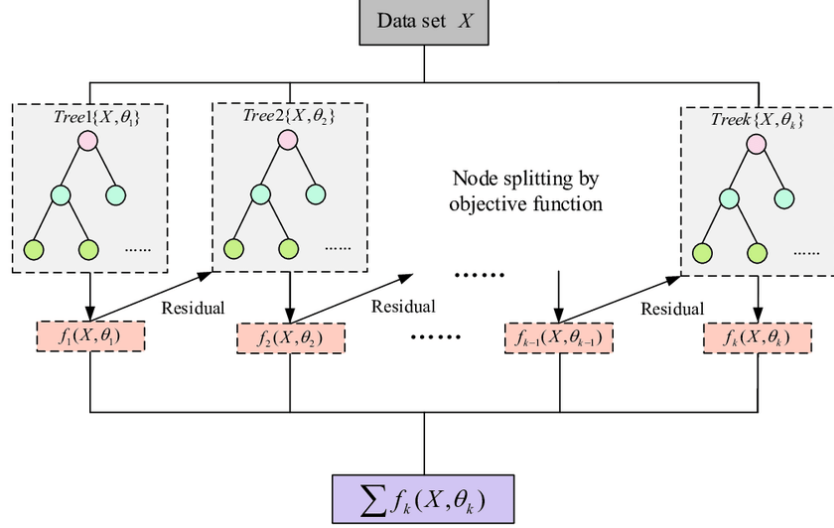


Figure 4: XGB Regressor

Mathematically, the model predicts:

$$\hat{y} = \sum_{k=1}^K f_k(x), \quad f_k \in \mathcal{F} \quad (12)$$

Where:

- f_k are the individual regression trees
- $\mathcal{F}$ is the space of all possible trees
- K is the number of boosting rounds

The objective is to minimize the following function:

$$\mathcal{L} = \sum_{i=1}^n \ell(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k) \quad (13)$$

Here:

- $\ell(y_i, \hat{y}_i)$ is a loss function (e.g., Mean Squared Error)
- $\Omega(f_k)$ is a regularization term to control model complexity

2.3.6 Light gradient boosting machine regressor

Light Gradient Boosting Machine (LightGBM) is a highly efficient, distributed gradient boosting framework developed by Microsoft for large-scale machine learning tasks such as classification, regression, and ranking. It builds upon the Gradient Boosting Decision Tree (GBDT) framework, employing a *leaf-wise* tree growth strategy instead of the conventional *level-wise* approach. Specifically, LightGBM grows the

tree by splitting the leaf with the maximum potential loss reduction, which accelerates convergence and often enhances model accuracy.

At each iteration m , LightGBM fits a regression tree $h_m(x)$ to the pseudo-residuals (negative gradients) of the loss function:

$$r_{im} = - \left[\frac{\partial L(y_i, F_{m-1}(x_i))}{\partial F(x_i)} \right] \quad (14)$$

where L is the loss function, $F_{m-1}(x)$ is the model prediction at the previous iteration, and y_i is the true label. The model is then updated as:

$$F_m(x) = F_{m-1}(x) + \nu \cdot h_m(x) \quad (15)$$

where $\nu \in (0, 1]$ is the learning rate that controls the contribution of each new tree.

LightGBM incorporates several algorithmic innovations to enhance scalability and efficiency. Gradient-based One-Side Sampling (GOSS) selectively retains data points with larger gradients to focus on harder-to-fit examples, while Exclusive Feature Bundling (EFB) reduces feature dimensionality by bundling mutually exclusive features. Furthermore, it uses histogram-based decision tree learning, where continuous features are discretized into bins, substantially reducing computation and memory usage.

These enhancements enable LightGBM to efficiently train on large-scale, high-dimensional datasets while achieving competitive predictive performance [17].

2.4 Metrics to evaluate machine learning algorithms

There are three commonly used metrics for evaluating predictions in regression-based machine learning problems:

2.4.1 Mean absolute error

The Mean Absolute Error (MAE) is the average of the absolute differences between the predicted and actual values. It quantifies the magnitude of the error without considering its direction (i.e., whether it is an overestimation or underestimation). An MAE value of 0 indicates a perfect prediction.

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i| \quad (16)$$

where $\hat{y}_i$ is the predicted value, y_i is the actual value, and N is the total number of observations.

2.4.2 Mean squared error and Root mean squared error

The Mean Squared Error (MSE) is similar to MAE in that it provides an estimate of the error magnitude. However, it penalizes larger errors more heavily due to squaring the residuals. The Root Mean Squared Error (RMSE) is the square root of the MSE and restores the error to the same units as the target variable, which is beneficial for interpretation and presentation. RMSE is also considered the standard deviation of the residuals (prediction errors). A lower RMSE indicates a better fit of the model.

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 \quad (17)$$

$$\text{RMSE} = \sqrt{\text{MSE}} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2} \quad (18)$$

2.4.3 R^2 metric

The R^2 metric, also known as the coefficient of determination, measures how well the regression model explains the variance in the dependent variable. An R^2 value close to 1 indicates a good fit, whereas a value close to 0 suggests a poor fit. It is computed as:

$$R^2 = 1 - \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{\sum_{i=1}^N (y_i - \bar{y})^2} \quad (19)$$

where $\bar{y}$ is the mean of the actual values.

These metrics collectively provide insight into the predictive accuracy and explanatory power of regression models.

2.5 SHapley Additive exPlanations

SHapley Additive exPlanations (SHAP) is a unified framework for interpreting predictions of machine learning models, grounded in cooperative game theory. It leverages the concept of Shapley values to quantify the marginal contribution of each feature to a model’s output. In this context, each feature of an input instance is considered a “player” in a cooperative game, and the predictive model serves as the “payout function.” The SHAP value for a feature represents its average contribution across all possible permutations of feature combinations. The formal expression for the Shapley value ϕ_i of feature i is given by:

$$\phi_i = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!(|N| - |S| - 1)!}{|N|!} [v(S \cup \{i\}) - v(S)] \quad (20)$$

Where:

- N is the set of all features
- S is a subset of N not containing i
- $v(S)$ is the model’s prediction given only the features in subset S

This formulation ensures that the SHAP values provide a fair distribution of contributions, satisfying properties such as efficiency, symmetry, dummy, and additivity.

SHAP offers several advantageous properties for interpretability: it is locally accurate (the sum of feature attributions equals the model output), consistent (increased model reliance on a feature does not reduce its attribution), and model-agnostic. As such, SHAP provides a theoretically sound and practically effective method for feature attribution, supporting model transparency and explainability across a wide range of applications[18].

2.6 Deep Learning and Embeddings

2.6.1 Deep Learning and Neural Networks

Deep learning is a subfield of machine learning that focuses on algorithms inspired by the structure and function of the human brain, known as artificial neural networks. These models are composed of multiple layers of interconnected nodes (neurons), where each layer transforms the data into increasingly abstract representations. The depth of a neural network—referring to the number of layers—enables it to model highly complex, non-linear relationships between input and output variables.

Neural networks learn these relationships by adjusting internal parameters (weights and biases) through a process called backpropagation, using optimization algorithms such as stochastic gradient descent or Adam. Deep learning has been successfully applied across domains, including image recognition, natural language processing, and time-series forecasting, due to its ability to automatically extract relevant features from raw data without requiring manual feature engineering.

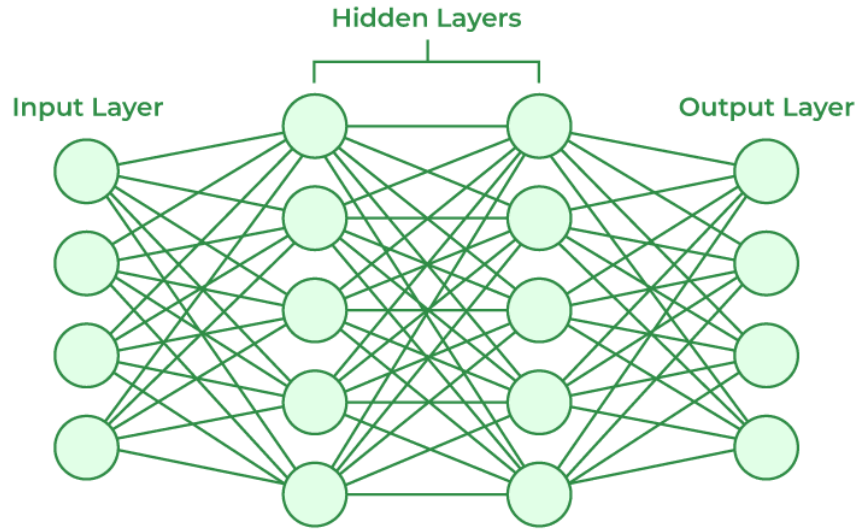


Figure 5: A basic neural network

2.6.2 Embeddings

Embeddings are low-dimensional, dense vector representations of high-dimensional or complex input data. In deep learning, embeddings are typically learned by neural networks during the training process. These vectors capture the most salient patterns and structures in the data, effectively compressing information into a compact form that is easier to work with for downstream tasks.

In practical applications, embeddings serve as intermediate representations that can be transferred to other models. For example, in natural language processing, word embeddings (like Word2Vec or BERT) capture semantic similarities between words. Similarly, in structured data tasks, embeddings derived from neural networks can encapsulate meaningful interactions between features, making them highly useful as inputs for traditional machine learning models such as KNN, XGBoost, or LightGBM.

Embeddings bridge the gap between raw data and model interpretability by transforming complex relationships into geometrically meaningful spaces. Their versatility and expressiveness make them a valuable tool in modern machine learning pipelines, particularly in settings where interpretability, transferability, or dimensionality reduction is required.

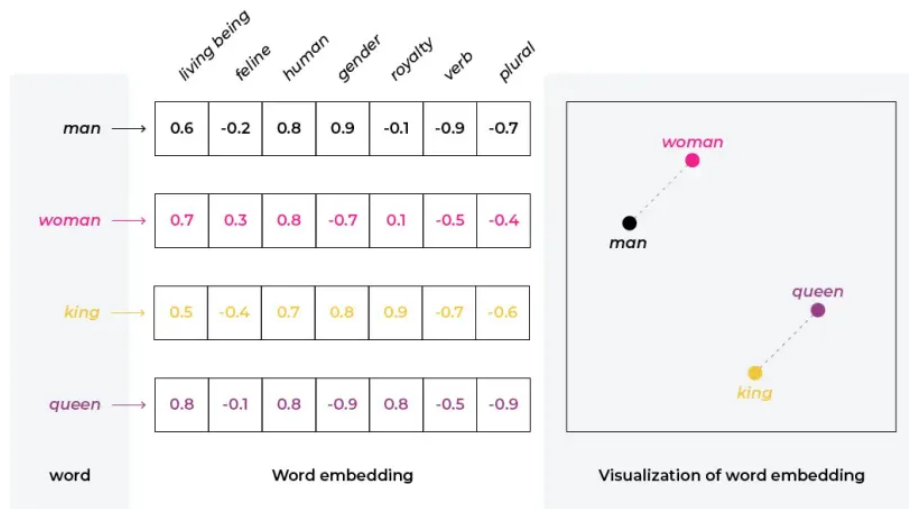


Figure 6: Dense embeddings generated by a neural network .

Chapter-III

Methodology and Result

3 Methodology

3.1 Dataset Details

This work utilizes a dataset derived from the publicly available battery degradation study published by Severson et al. in “*Data-driven prediction of battery cycle life before capacity degradation.*” The original dataset comprises 124 commercial lithium iron phosphate (LFP)/graphite lithium-ion cells (A123 Systems, model APR18650M1A) subjected to fast-charging protocols until end-of-life. These cells were tested using a 48-channel Arbin LBT potentiostat within a temperature-controlled chamber maintained at 30 °C, ensuring standardized testing conditions. Each cell has a nominal voltage of 3.3 V and a nominal capacity of 1.1 Ah.

For structured analysis, the dataset was divided into three batches based on the date of test initiation:

- **Batch 1:** 2017-05-12
- **Batch 2:** 2017-06-30
- **Batch 3:** 2018-04-12

Each batch contains 40–48 cells. Notably, four cells from Batch 1 continued into Batch 2. After excluding batteries that did not meet the 80% capacity retention threshold, the filtered data contains 41 cells from Batch 1, 43 from Batch 2, and 40 from Batch 3 [19].

To facilitate further analysis, the data was preprocessed and stored in Python-readable format. While the original files were in MATLAB `.mat` format, converted versions in `.pkl` format allowed easy manipulation via Python’s `pickle` module. These were subsequently compiled into nested dictionaries and later structured into dataframes using the `Pandas` library.

3.2 Data Exploration

To understand the underlying behavior and degradation patterns of lithium-ion batteries, an exploratory data analysis (EDA) was conducted on the dataset comprising operational data from 124 commercial lithium iron phosphate (LFP)/graphite cells. The dataset, originally curated by Severson et al., is structured into three batches—Batch1, Batch2, and Batch3—each consisting of approximately 2,300 to 2,400 rows representing measurements taken across successive charge-discharge cycles.

Each row in the dataset contains the following features: internal resistance (IR), charge capacity (QC), discharge capacity (QD), average temperature (Tavg), minimum and maximum temperatures (Tmin and Tmax), charge time, cycle number, battery identifier (battery_id), and the total cycle life of the battery (cycle_life). Batch1 also includes additional metadata such as voltage thresholds and reference capacities (C1, Q1, and C2).

The cycle life of the batteries ranges significantly across the three batches. Batch1 includes cells with life spans ranging from 600 to 1,200 cycles, Batch2 comprises shorter-lived cells ranging from 300 to 700 cycles, while Batch3 contains cells with extended life spans exceeding 1,800 cycles. This variation across batches offers a diverse basis for analysis and model training.

Initial statistical summaries and visual plots were used to examine the evolution of critical features such as IR, charge time, and temperature over cycling. As expected, the internal resistance demonstrated a monotonic increase with cycle number across all batches, indicating natural cell aging. However, the rate of increase was notably slower in Batch3, suggesting better long-term stability. In contrast, Batch2 exhibited more rapid degradation, with elevated resistance values emerging within the first 50 cycles.

Charge time showed similarly discriminative trends. Batch 3 displayed smooth and stable charge time curves across the first 50 cycles, indicative of healthy battery behavior. Batch 2, on the other hand, revealed irregular and fluctuating charge times, possibly due to early inefficiencies or manufacturing inconsistencies. Average temperatures across all batches remained within the expected operational range (30–33°C), but Batch 2 showed signs of a gradual thermal rise, further indicating poorer thermal efficiency and faster degradation.

3.2.1 Batch-1 Analysis

Batch 1 comprises operational data from 41 lithium-ion battery cells, each measured over their first 50 charge-discharge cycles. This batch reflects moderate-performing cells, with cycle lives ranging from approximately 600 to 1,200 cycles. Upon visual inspection of the Batch 1 data Figure 7, it is observed that the internal resistance (IR) increases steadily with cycling, aligning with typical degradation patterns. Charge and discharge capacities remain relatively stable during the early cycles, indicating consistent electrochemical behavior. Charge time shows a mild upward trend, signifying a gradual decline in charging efficiency as the battery ages. Temperature values (Tavg, Tmin, Tmax) across the 50 cycles remain within a narrow range of 30°C to 32.5°C, suggesting thermal stability and controlled testing conditions. Importantly, the dataset for Batch1 includes additional attributes—namely C1, Q1, and C2—which represent voltage and capacity thresholds useful for feature engineering in predictive modeling. The data is clean, uniformly structured, and exhibits gradual aging behavior, making Batch1 highly suitable for training baseline regression models and for studying average-performing batteries.

	IR	QC	QD	Tavg	Tmin	Tmax	\
0	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	
1	0.016742	1.071042	1.070689	31.875011	29.566130	35.652016	
2	0.016724	1.071674	1.071900	31.931490	29.604385	35.692978	
3	0.016681	1.072304	1.072510	31.932603	29.744202	35.680588	
4	0.016662	1.072970	1.073174	31.959322	29.644709	35.728691	
...	...	...	...	...	...	...	
2295	0.016683	1.087311	1.087584	32.831865	29.864557	36.536152	
2296	0.016668	1.087259	1.087567	32.868466	29.732851	36.546894	
2297	0.016663	1.087236	1.087492	32.828632	29.857639	36.498089	
2298	0.016675	1.087248	1.087526	32.865756	29.840807	36.602509	
2299	0.016682	1.087206	1.087513	32.886698	29.785046	36.635803	
	chargetime	cycle	battery_id	cycle_life	C1	Q1	C2
0	0.000000	1.0	b1c0	1190.0	3.6	80.0	3.6
1	13.341250	2.0	b1c0	1190.0	3.6	80.0	3.6
2	13.425777	3.0	b1c0	1190.0	3.6	80.0	3.6
3	13.425167	4.0	b1c0	1190.0	3.6	80.0	3.6
4	13.341442	5.0	b1c0	1190.0	3.6	80.0	3.6
...	...	...	...	...	...	...	...
2295	10.220962	46.0	b1c45	599.0	8.0	35.0	3.6
2296	10.304470	47.0	b1c45	599.0	8.0	35.0	3.6
2297	10.220978	48.0	b1c45	599.0	8.0	35.0	3.6
2298	10.221937	49.0	b1c45	599.0	8.0	35.0	3.6
2299	10.305015	50.0	b1c45	599.0	8.0	35.0	3.6
[2300 rows x 13 columns]							

Figure 7: Batch-1 Data Set

The Figure 8 is analysis involved plotting the discharge capacity (QD) of the lithium-ion battery cell b1c43 against the number of charge-discharge cycles using Python's matplotlib library. The graph shows how the discharge capacity decreases progressively as the cycle number increases, illustrating the typical degradation behavior of the battery over time. The resulting plot clearly visualizes the loss of capacity with cycling, providing insight into the battery's aging characteristics and enabling assessment of its remaining useful life.

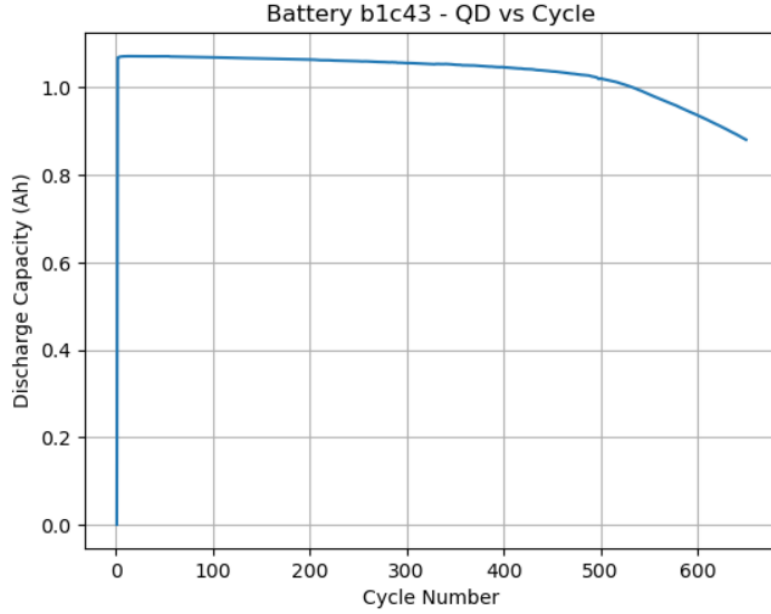


Figure 8: QD vs Cycle

Figure 9 analyze the voltage response during a specific discharge cycle, the voltage (V) of battery cell b1c43 was plotted against the discharge capacity (Qd) for cycle 10. This visualization shows how the voltage decreases progressively as the battery discharges, reflecting the typical discharge curve of a lithium-ion cell. The resulting plot provides insight into the electrochemical behavior and energy delivery characteristics of the battery during this cycle.

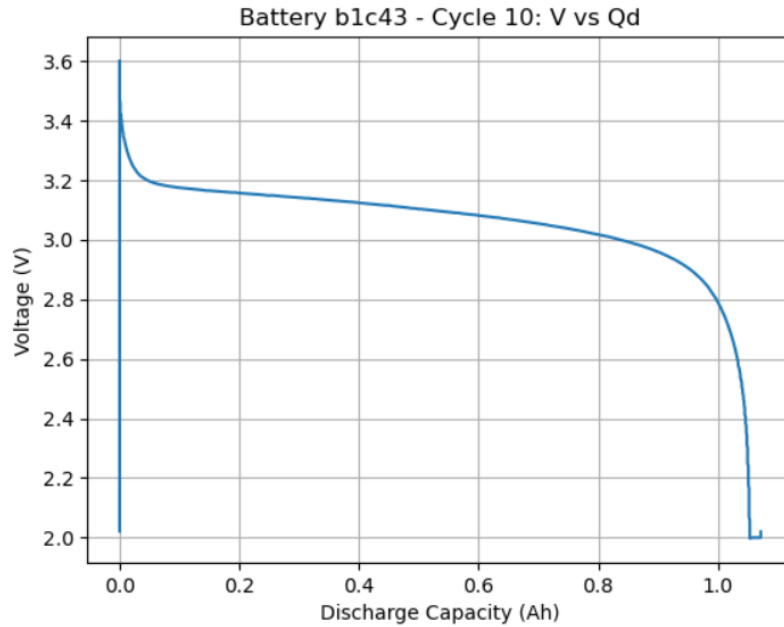


Figure 9: Cycle 10: V vs Qd

The Figure 10 is a three-dimensional scatter plot was generated to explore the relationship between the measured parameters C1 and Q1 and the cycle life of the batteries in the Batch1 dataset. In this visualization, C1 and Q1 are plotted along the X and Y axes, respectively, while cycle life is represented on the Z-axis and also encoded by point color using the Viridis colormap. This 3D plot enables the identification of trends, clusters, or outliers that may indicate how these features influence battery longevity.

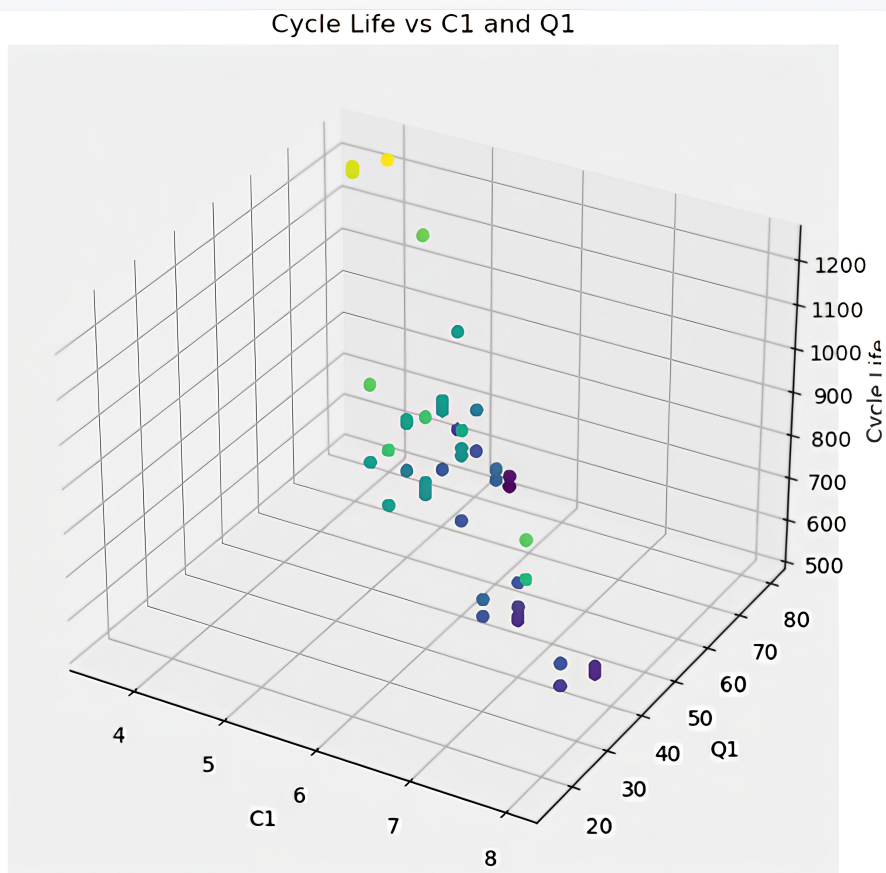


Figure 10: 3D Graph of cycle life vs C1 and Q1

Figure 11 is a two-dimensional scatter plot was created to examine the relationship between the C1 parameter and battery cycle life, while encoding the Q1 values as color using the Plasma colormap. In this visualization, C1 is plotted on the X-axis and cycle life on the Y-axis, with each point colored according to its corresponding Q1 measurement and sized uniformly for clarity. A colorbar legend was included to interpret the Q1 scale, and grid-lines were added to improve readability. This plot enables the identification of trends or clusters that may reveal how variations in C1 and Q1 jointly affect battery longevity.

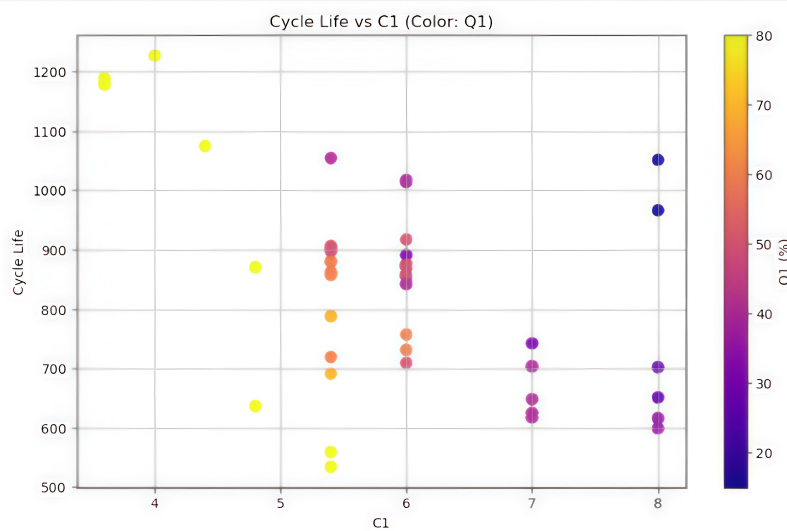


Figure 11: Cycle life vs C1

Figure 12 is procedure implements a structured visualization framework to assess the temporal dynamics of multiple battery parameters across sequential charge–discharge cycles. Initially, all measured variables excluding the cycle identifier were extracted for analysis. The total number of variables determined the subplot grid configuration, with three plots allocated per row to optimize readability and comparative clarity. For each parameter, an individual subplot was generated, depicting its progression as a function of cycle count. Descriptive annotations, including axis labels, legends, and titles, were incorporated to facilitate interpretation, while grid lines enhanced visual precision. The resulting composite figure provides an integrated overview of parameter evolution over time, enabling simultaneous examination of trends such as gradual impedance decline (as observed in IR), relative stabilization of charge capacities (QC and QD), and fluctuating thermal behavior (Tavg, Tmin, Tmax). In contrast, parameters like battery ID remained invariant, confirming consistent identification throughout the cycles. Collectively, these visualizations support comprehensive comparative analysis of degradation patterns and operational stability across all monitored features.

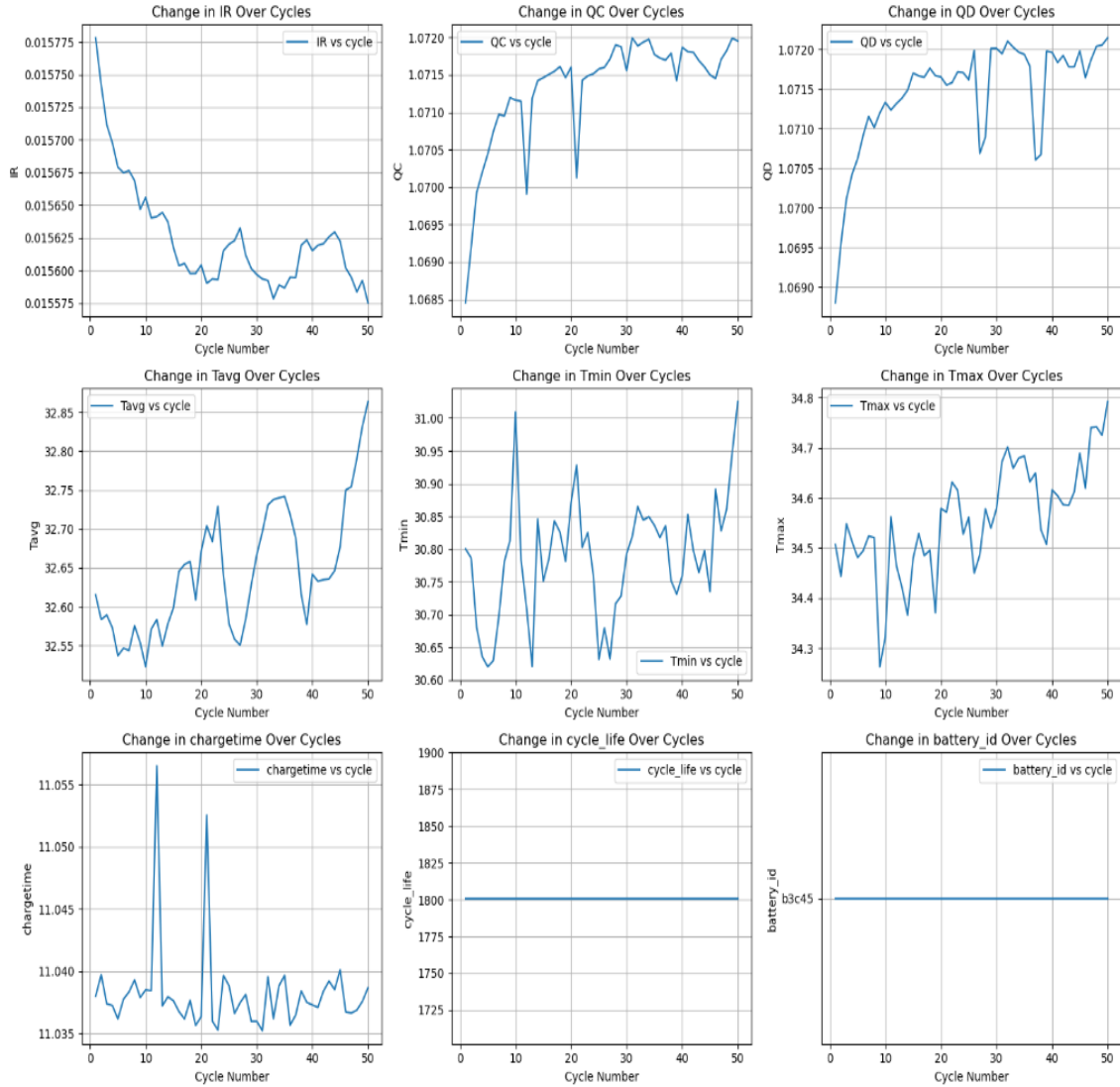


Figure 12: Comparison of very feature with cycle number

Figure 13 shows the analysis of battery degradation across cycles is done by plotting key performance indicators with discharge capacity (QD), internal resistance (IR), average temperature (Tavg), and charging time—for each cell in a dataset respectively. Cycle-wise summary data is extracted for the specified feature from each battery's log and visualizes it across all cells, enabling comparative trend analysis. The resulting plots reveal consistent patterns of degradation: QD generally declines over time, reflecting capacity fade, while IR exhibits a gradual increase with sudden jumps around 800 cycles, sug-

gesting aging-related resistance growth or failure events. T_{avg} remains relatively stable in early cycles but shows heightened fluctuation and thermal instability beyond 400–800 cycles. Similarly, charging time remains constant initially but spikes in later stages, potentially due to rising resistance or thermal limitations. These visualizations support the identification of anomalous behaviors and performance divergence among cells, offering valuable insights for battery health diagnostics and life-cycle modeling.

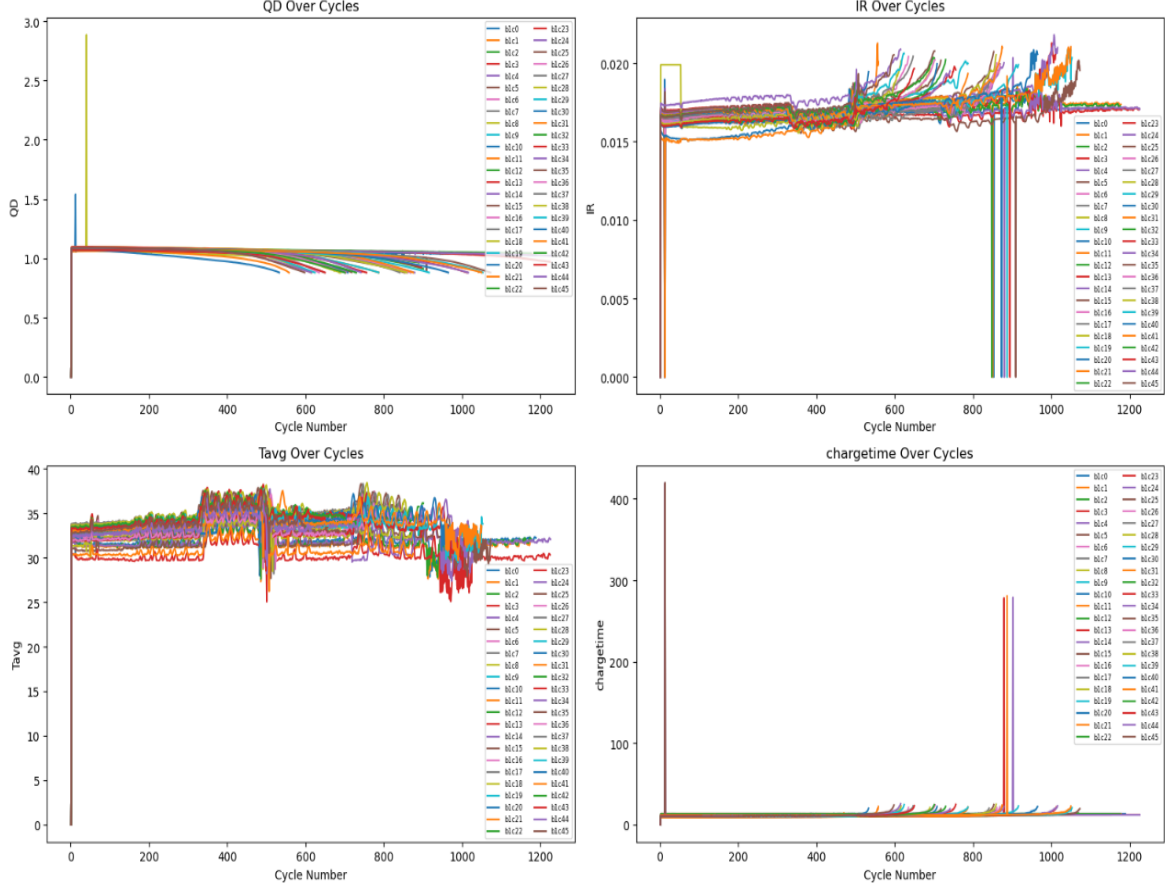


Figure 13: Analysis of battery degradation across cycle done by plotting key performance indicators: discharge capacity (QD), internal resistance (IR), average temperature (T_{avg}), and charging time for each cell in a dataset respectively

3.2.2 Batch-2 Analysis

The Batch-2 dataset was constructed by processing raw battery cycling data stored in the file[2017-06-30][19]. This file contains measurements of multiple battery cells, organized in hierarchical MATLAB structures accessed using the `h5py` library. Initially, all cells were iterated over to extract both summary-level and cycle-level data for each cell. Summary information included key parameters such as internal resistance (IR), charge capacity (QC), discharge capacity (QD), average, minimum, and maximum temperatures, charge time, and the corresponding cycle index. For each cell, these values were assembled into structured dictionaries along with metadata such as cycle life and charging policy.

To facilitate downstream analysis and visualization, individual DataFrames were created for each cell, incorporating all summary parameters and appending columns indicating the battery identifier and total cycle life. These DataFrames were then concatenated into a single combined DataFrame containing the complete record of all cycles and all cells in Batch 2. In order to focus the study on early degradation behavior, an additional filtering step was applied to retain only records corresponding to the first 50 cycles for each battery. The final dataset, referred to as Batch 2 (Figure 14), thus provides a consolidated, cycle-resolved view of the initial performance characteristics of all cells in Batch 2, enabling systematic comparison of parameter trends and the identification of early indicators of capacity fade or resistance growth.

	IR	QC	QD	Tavg	Tmin	Tmax	\
0	0.017542	1.067561	1.068057	30.363545	28.999689	31.487127	
1	0.017505	1.069810	1.070054	29.160560	26.734909	31.661930	
2	0.017571	1.071287	1.071910	29.711384	28.086988	31.111683	
3	0.017263	1.072626	1.072965	30.235232	28.464998	31.539034	
4	0.017489	1.071483	1.071910	29.481769	27.695206	31.414228	
...	...	...	...	...	...	...	
2395	0.017171	1.049489	1.049833	32.497725	29.385515	36.459629	
2396	0.017159	1.049492	1.049915	32.435424	29.585482	36.574768	
2397	0.017161	1.049419	1.049806	32.397096	29.739912	36.586670	
2398	0.017146	1.049345	1.049750	32.311683	29.463625	36.295872	
2399	0.017180	1.049345	1.049750	32.314531	29.546947	36.382740	
	chargetime	cycle	battery_id	cycle_life	C1	Q1	C2
0	10.332668	1.0	b2c0	300.0	1.0	4.0	6.0
1	10.180050	2.0	b2c0	300.0	1.0	4.0	6.0
2	10.308322	3.0	b2c0	300.0	1.0	4.0	6.0
3	10.276183	4.0	b2c0	300.0	1.0	4.0	6.0
4	10.263642	5.0	b2c0	300.0	1.0	4.0	6.0
...	...	...	...	...	...	...	...
2395	10.179140	46.0	b2c47	713.0	6.0	60.0	3.0
2396	10.179088	47.0	b2c47	713.0	6.0	60.0	3.0
2397	10.179405	48.0	b2c47	713.0	6.0	60.0	3.0
2398	10.096282	49.0	b2c47	713.0	6.0	60.0	3.0
2399	10.179120	50.0	b2c47	713.0	6.0	60.0	3.0

[2400 rows x 13 columns]

Figure 14: Batch 2 Data

The Figure 15 shows graphical representation of discharge capacity degradation of multiple lithium-ion battery cells was evaluated across their operational life cycles. For each cell in the dataset, discharge capacity values (measured in ampere-hours) were plotted against the number of charge-discharge cycles. The figure illustrates individual capacity fade trajectories for all battery cells, enabling direct visual comparison of performance loss over time. Most cells show a clear downward trend in discharge capacity, indicative of typical aging mechanisms such as loss of active lithium, electrolyte decomposition, or structural degradation of electrode materials. Notably, some cells exhibit sharp drops or irregular spikes, particularly around 200–300 cycles, suggesting the presence of anomalies, data logging artifacts, or early-stage failure events. A few cells sustain higher capacities beyond 800 cycles, highlighting potential differences in manufacturing quality, usage conditions, or cell-to-cell variability. This visualization provides a comprehensive overview of degradation behavior and is instrumental for identifying outliers, estimating end-of-life thresholds, and informing predictive maintenance strategies in battery management systems.

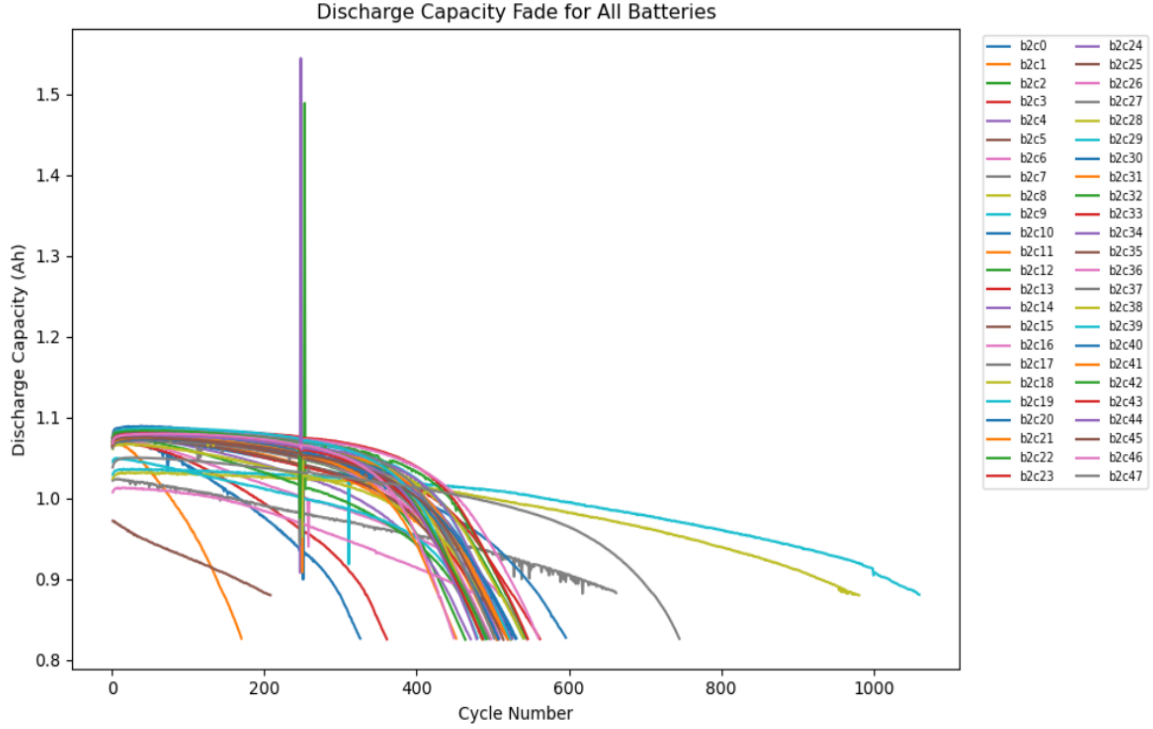


Figure 15: Discharge Capacity Fade for All Batteries

3.2.3 Batch 3 Analysis

The Batch 3 dataset was generated by processing raw battery cycling data contained in the file 2018-04-12 [19]. This file includes detailed measurements of multiple cells organized as hierarchical MATLAB structures, which were accessed programmatically using the `h5py` library in Python. For each cell in Batch 3, summary-level parameters were extracted, including internal resistance (IR), charge capacity (QC), discharge capacity (QD), average, minimum, and maximum temperatures, charge time, and the corresponding cycle indices. These data were stored in structured dictionaries alongside metadata such as total cycle life and the applied charge policy.

Additionally, full cycle-resolved records were retrieved for each cell, encompassing current, voltage, charge and discharge profiles, temperature trajectories, and derived quantities such as dQ/dV . To enable comparative analysis and visualization, individual pandas DataFrames were constructed for each battery, incorporating all summary measurements and appending battery identifiers and cycle life annotations. These data frames were concatenated into a single combined dataset capturing the complete measurement history of Batch 3.

Subsequently, to focus the analysis on early-cycle behavior, a filtering procedure was applied to retain only observations corresponding to the first 50 cycles of each cell. The resulting DataFrame, designated Batch 3, provides a consolidated representation of the initial performance characteristics of all batteries in this batch and forms the basis for subsequent statistical and visual analyses.

	IR	QC	QD	Tavg	Tmin	Tmax	\
0	0.015435	1.066090	1.066573	32.149495	30.830272	33.433834	
1	0.015399	1.066964	1.067455	32.106606	30.779943	33.403964	
2	0.015365	1.067611	1.068285	32.088511	30.824746	33.325268	
3	0.015326	1.068164	1.068708	32.086306	30.804544	33.304202	
4	0.015309	1.068695	1.069094	32.041147	30.791834	33.257191	
...	...	...	...	...	...	...	
49252	0.015602	1.071451	1.071640	32.750094	30.891405	34.619160	
49253	0.015595	1.071707	1.071866	32.754391	30.828009	34.740250	
49254	0.015583	1.071824	1.072039	32.790772	30.861229	34.741764	
49255	0.015592	1.071991	1.072053	32.832211	30.947521	34.724735	
49256	0.015575	1.071953	1.072145	32.863722	31.025038	34.791729	
	chargetime	cycle	battery_id	cycle_life	C1	Q1	C2
0	10.043408	1.0	b3c0	1009.0	5.0	67.0	4.0
1	10.043255	2.0	b3c0	1009.0	5.0	67.0	4.0
2	10.042333	3.0	b3c0	1009.0	5.0	67.0	4.0
3	10.052677	4.0	b3c0	1009.0	5.0	67.0	4.0
4	10.042382	5.0	b3c0	1009.0	5.0	67.0	4.0
...	...	...	...	...	...	...	...
49252	11.036730	46.0	b3c45	1801.0	4.8	80.0	4.8
49253	11.036633	47.0	b3c45	1801.0	4.8	80.0	4.8
49254	11.036873	48.0	b3c45	1801.0	4.8	80.0	4.8
49255	11.037575	49.0	b3c45	1801.0	4.8	80.0	4.8
49256	11.038668	50.0	b3c45	1801.0	4.8	80.0	4.8

[2300 rows x 13 columns]

Figure 16: Batch 3 Data

3.2.4 Merged Batch Dataset

In this study, the datasets from Batch 1, Batch 2, and Batch 3 were combined into a single unified DataFrame named `total_df`. The merging process was executed using the `pd.concat()` function from the `pandas` library, which vertically concatenates the individual batch DataFrames into one comprehensive structure. Following the merge, the dataset was cleaned by removing all rows containing null values to ensure data consistency and reliability for subsequent analysis. The cleaned and merged dataset was saved. This dataset serves as the foundational input for all downstream machine learning workflows, including preprocessing, feature engineering, and model development. The dataset comprises approximately 7,000 rows and 13 columns, encompassing critical battery-level and cycle-specific features such as internal resistance, charge and discharge capacities, cycle life, average temperature metrics, and engineered features like C1, Q1, and C2.

This integrated dataset was used to train and evaluate several regression models—namely, Linear Regression, LightGBM Regressor, Decision Tree Regressor, and K-Nearest Neighbors Regressor—for the prediction of battery cycle life. Each model was trained on preprocessed (quantile-transformed) data and evaluated using key performance metrics, including Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), Coefficient of Determination (R^2), and Average Percentage Error (APE). The merged dataset structure supports robust statistical analysis, performance visualization, and effective generalization across varying battery degradation profiles.

	IR	QC	QD	Tavg	Tmin	Tmax	chargetime	cycle	battery_id	C1	Q1	C2	cycle_life
0	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	1.0	b1c0	3.6	80.0	3.6	1190.0
1	0.016742	1.071042	1.070689	31.875011	29.566130	35.652016	13.341250	2.0	b1c0	3.6	80.0	3.6	1190.0
2	0.016724	1.071674	1.071900	31.931490	29.604385	35.692978	13.425777	3.0	b1c0	3.6	80.0	3.6	1190.0
3	0.016681	1.072304	1.072510	31.932603	29.744202	35.680588	13.425167	4.0	b1c0	3.6	80.0	3.6	1190.0
4	0.016662	1.072970	1.073174	31.959322	29.644709	35.728691	13.341442	5.0	b1c0	3.6	80.0	3.6	1190.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...
6995	0.015602	1.071451	1.071640	32.750094	30.891405	34.619160	11.036730	46.0	b3c45	4.8	80.0	4.8	1801.0
6996	0.015595	1.071707	1.071866	32.754391	30.828009	34.740250	11.036633	47.0	b3c45	4.8	80.0	4.8	1801.0
6997	0.015583	1.071824	1.072039	32.790772	30.861229	34.741764	11.036873	48.0	b3c45	4.8	80.0	4.8	1801.0
6998	0.015592	1.071991	1.072053	32.832211	30.947521	34.724735	11.037575	49.0	b3c45	4.8	80.0	4.8	1801.0
6999	0.015575	1.071953	1.072145	32.863722	31.025038	34.791729	11.038668	50.0	b3c45	4.8	80.0	4.8	1801.0

7000 rows × 13 columns

Figure 17: Total_Batch

3.2.5 Data Cleaning, Cycle Selection, and Imputation

Following the initial merge, the dataset underwent a systematic cleaning and filtering process to ensure quality and consistency of the data used for battery life prediction. First, an outlier detection step was performed on key features—including internal resistance (IR), charge and discharge capacities (QC, QD), temperature statistics (Tavg, Tmin, Tmax), and charge time—using the InterQuartile Range (IQR) method with an aggressive multiplier of 2.4 to exclude anomalous readings. This step removed rows containing extreme values that could skew the regression models.

Next, only batteries with at least 20 full charge–discharge cycles were retained, filtering out under-represented battery IDs that could introduce noise or instability during training. For each valid battery, a maximum of 300 initial cycles were preserved to ensure a balanced representation of early degradation patterns across all batteries. This resulted in a compact yet information-rich dataset ideal for training generalizable models.

To address remaining missing values, K-Nearest Neighbors (KNN) imputation with $k = 5$ was applied to a selected subset of columns, including both raw sensor readings and derived features such as C1, Q1, and C2. This imputation method leverages the similarity between battery cycles to infer realistic values, improving data completeness without introducing synthetic bias. The resulting cleaned and imputed DataFrame provides a robust foundation for all downstream modeling tasks, ensuring both data integrity and statistical reliability.

3.2.6 Feature Aggregation and Quantile-Transformed Regression Pipeline

To enable robust cycle life prediction using engineered features, a battery-level aggregation pipeline was constructed from the preprocessed per-cycle dataset. For each `battery_id`, statistical descriptors such as mean, standard deviation, minimum, maximum, skewness, and kurtosis were computed across key cycle-wise features: internal resistance (IR), charge capacity (QC), discharge capacity (QD), and average temperature (Tavg). Additionally, linear trends were computed via least-squares regression across cycles to capture degradation trajectories.

To incorporate electrochemical behavior, a domain-inspired feature set based on differential capacity analysis ($\Delta Q/\Delta V$) was introduced. Specifically, capacity readings at cycles 10 and 100 were interpolated across a fixed voltage grid (2.5–4.0 V), and the resulting ΔQ vector was summarized using mean, variance, skewness, and kurtosis to quantify changes in battery response over early aging.

Histogram-based descriptors were also extracted from IR and Tavg distributions using 5-bin density-normalized histograms, providing shape-based information about their variation over cycles. The final aggregated dataset consisted of one row per battery, with each row representing a comprehensive statistical fingerprint of its early-cycle behavior.

After aggregation, missing values were imputed using the Iterative Imputer to maintain inter-feature consistency. A Quantile Transformer was then applied independently to both features and target variable (cycle life), ensuring that all variables followed a uniform distribution—which improves training stability for tree-based regressors. The processed dataset was split into training and testing subsets using an 80–20 ratio, and the resulting transformations were saved via `joblib` for reproducibility in downstream modeling.

3.2.7 Cycle Life Prediction Using Quantile-Transformed XGBoost with Calibration

In this study, a data-driven machine learning approach was employed to predict the cycle life of lithium-ion batteries using features derived from the first 50 charge-discharge cycles. The dataset included variables such as internal resistance (IR), charge capacity (QC), discharge capacity (QD), and temperature metrics (Tavg, Tmin, Tmax), recorded across multiple battery cells. The initial data underwent outlier removal using an interquartile range (IQR)-based filter, and K-Nearest Neighbors (KNN) imputation was used to fill in missing values. Only batteries with a minimum of 20 complete cycles were retained for model development to ensure robustness.

Feature engineering was performed at the battery level, incorporating both statistical descriptors (mean, standard deviation, skewness, kurtosis, etc.) and trend-based metrics calculated over the 50-cycle span. In addition, domain-specific Q/V (delta capacity over voltage) profiles between cycles 10 and 100 were included to capture electrochemical signatures associated with early degradation. Histogram features were also derived from IR and temperature distributions to enhance variability representation.

All features and target values were normalized using quantile transformation to reduce skewness and improve model performance. A regression model using the XGBoost algorithm was then trained on the transformed data. To further refine predictions, calibration was applied using Centered Isotonic Regression (CIR), which helped align predicted cycle life values with their true distribution. The final predictions were converted back to the original scale using inverse quantile transformation.

The model’s performance was evaluated using standard metrics, yielding a Mean Absolute Error (MAE) of 160.9 cycles, Root Mean Squared Error (RMSE) of 239.1 cycles, an R^2 score of 0.496, and an Average Percentage Error (APE) of 17.2%. These results indicate a moderate but informative predictive capability.

The results are visually represented in the accompanying scatter plot titled XGB Predicted vs Actual Cycle Life (Figure18), which illustrates the relationship between the model’s predicted cycle life and the actual cycle life of the batteries. Each point represents an individual battery, with the red dashed line indicating the ideal case where prediction matches the actual value. The majority of points cluster near the diagonal, particularly in the low-to-mid cycle life range, demonstrating that the model achieves reasonable accuracy in most cases, though deviations become more pronounced for batteries with longer lifespans.

This predictive framework highlights the value of early-cycle data combined with robust statistical and electrochemical feature extraction, normalization, and calibration, providing a foundation for early prediction of battery end-of-life in real-world applications.

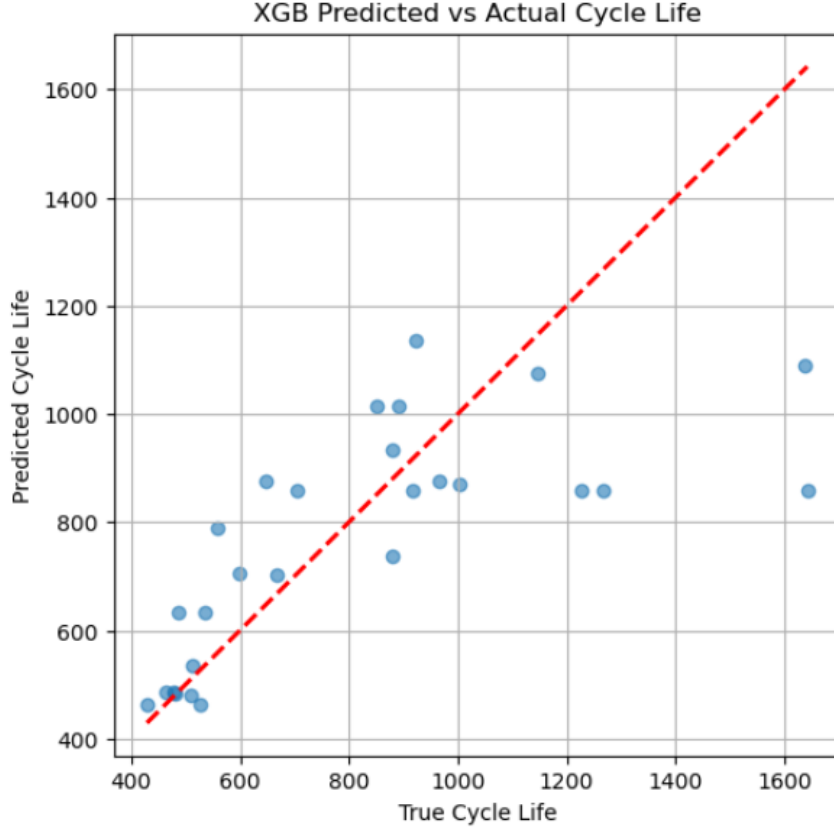


Figure 18: K-Nearest Neighbors

3.2.8 Cycle Life Prediction Using LightGBM with Quantile Normalization and Calibration

Training the LightGBM model on the quantile-transformed dataset, the predictions were calibrated using Centered Isotonic Regression (CIR), which corrected for any monotonic deviations between predicted and actual target distributions. This post-processing step enhanced the reliability of the predicted values by enforcing a non-decreasing relationship and reducing systematic prediction bias.

The model's performance was evaluated using multiple metrics: it achieved a Mean Absolute Error (MAE) of 160.9 cycles, a Root Mean Squared Error (RMSE) of 239.1 cycles, an R^2 score of 0.496, and an Average Percentage Error (APE) of 17.2%. These results indicate that while the model does not perfectly capture the full variability in cycle life, it provides a reasonably accurate estimation based on early-cycle features.

The output Figure 19 visually illustrates the model's performance. In the plot, each blue dot represents a test battery cell, with the x-axis showing the true cycle life and the y-axis showing the predicted cycle life from the LightGBM model. The red dashed diagonal line represents the ideal 1:1 prediction line, where predictions would perfectly match the actual values. Most of the points lie relatively close to this reference line, demonstrating the model's reasonable accuracy. However, some deviation—particularly at higher cycle life ranges—suggests room for improvement, potentially through more complex calibration or feature expansion.

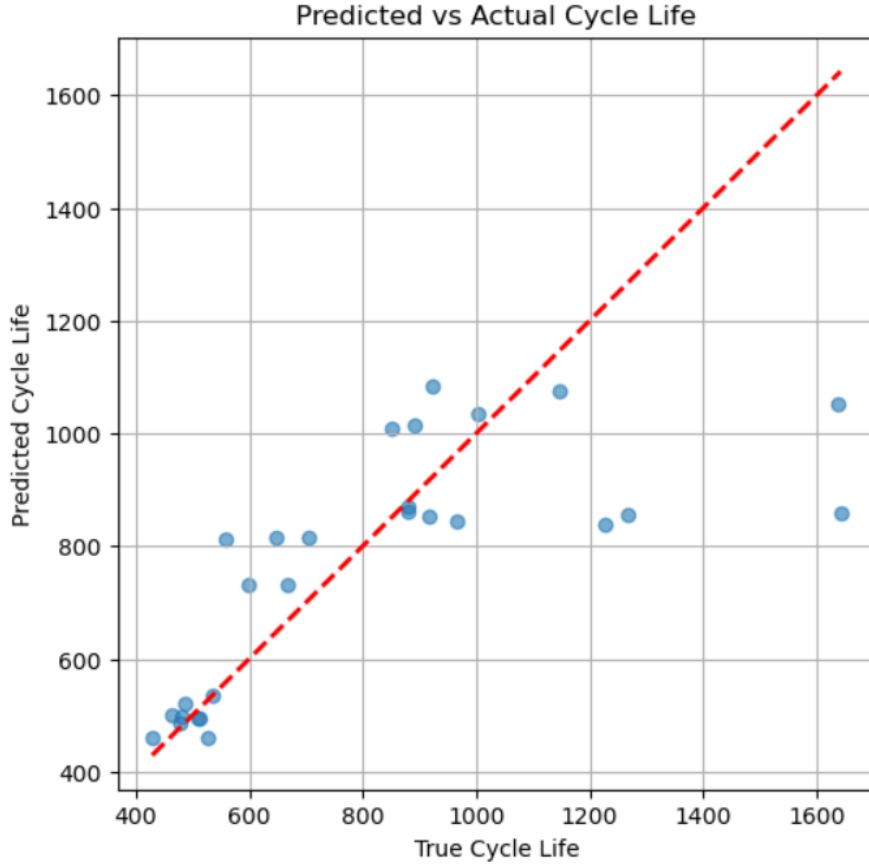


Figure 19: Predicted vs Actual Cycle Life

3.2.9 Cycle Life Prediction Using Decision Tree Regression with Calibration

A Decision Tree Regressor was employed to predict the cycle life of lithium-ion batteries using early-stage charging data. To ensure a more uniform and normalized feature distribution, a quantile transformation was applied to both the input features and target cycle life values. This helped the model better capture underlying patterns that might otherwise be masked by skewed distributions. To improve the alignment of the model predictions with the actual outcomes, Centered Isotonic Regression (CIR) was applied as a calibration technique. CIR ensures the monotonicity of predictions and is especially useful in reducing overfitting in decision tree models. After calibration, predictions were inverse-transformed to return them to their original scale, allowing for accurate interpretation and evaluation.

The model's performance was assessed using standard regression metrics, achieving a Mean Absolute Error (MAE) of 212.2 cycles, Root Mean Squared Error (RMSE) of 302.8 cycles, an R^2 score of 0.192, and an Average Percentage Error (APE) of 24.3%. These results indicate that while the decision tree model captures some signal, it has lower predictive accuracy compared to more sophisticated models like LightGBM.

The accompanying output Figure 20 illustrates the performance of the Decision Tree model. Each blue point represents an individual battery's predicted vs actual cycle life. The x-axis denotes the true cycle life, while the y-axis reflects the model's predictions. The red dashed diagonal line represents the ideal scenario where predicted and true values are equal. The noticeable spread around this reference line suggests that the model's predictions are less consistent, with larger deviations at both lower and higher cycle life values, underscoring the model's limited generalization capacity compared to ensemble methods.

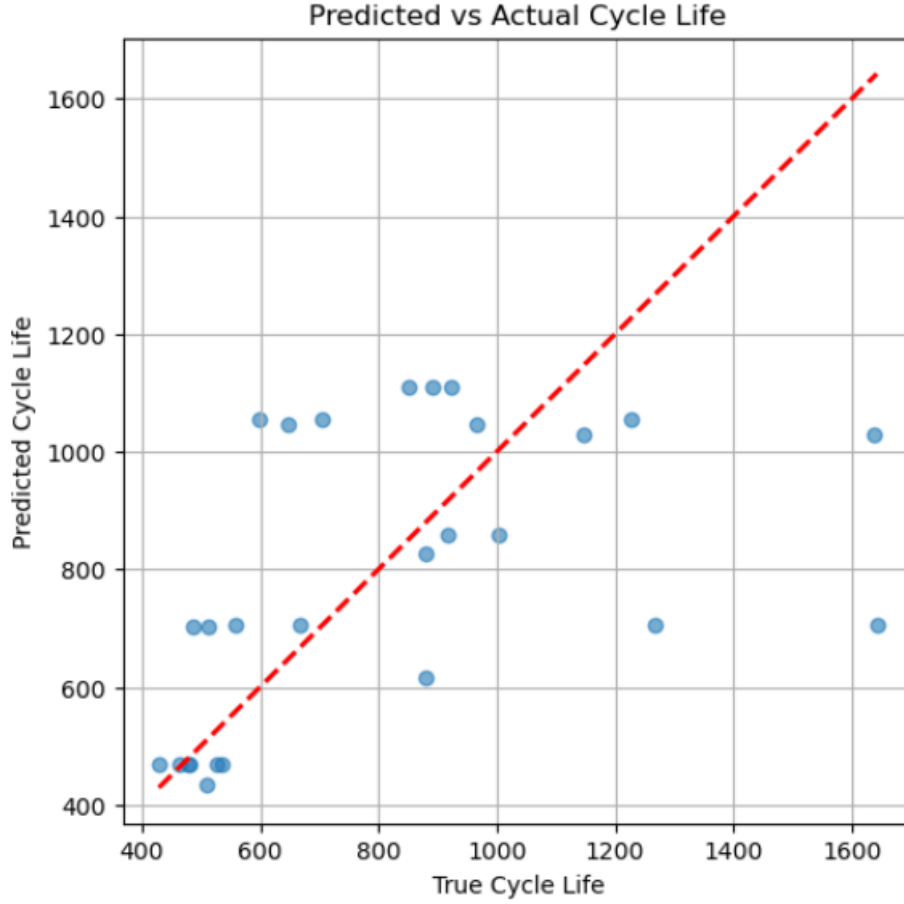


Figure 20: Decision Tree predictions

Figure 21 illustrates the structure of the trained Decision Tree Regressor with a maximum depth of 4, developed to predict battery cycle life using quantile-transformed features derived from early-cycle data. Each node in the tree represents a decision point based on a threshold value of a particular input feature. The boxes display key information including the feature used for the split, the number of samples reaching that node, the average predicted value (in quantile-transformed scale), and the associated squared error (impurity). The tree recursively splits the data to minimize prediction error, effectively partitioning the input space into regions with similar output values. This structure not only provides competitive performance but also offers clear interpretability, making it suitable for understanding how specific early-cycle measurements contribute to battery lifetime estimation. Such a visual representation is valuable in domains where model transparency is essential for validation and decision-making.

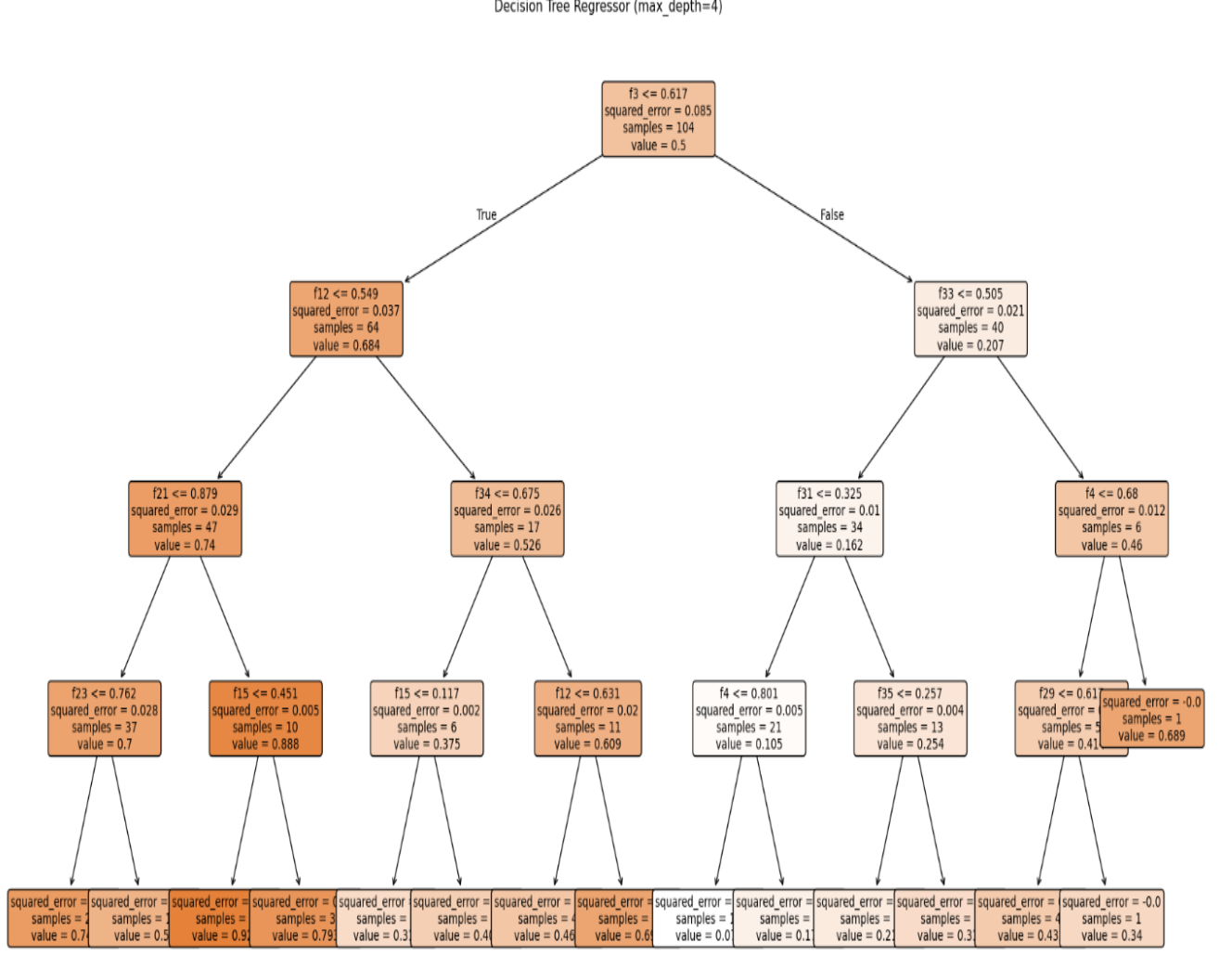


Figure 21

3.2.10 Cycle Life Prediction Using K-Nearest Neighbors Regression

In this implementation, the K-Nearest Neighbors (KNN) algorithm was used to predict the cycle life of lithium-ion batteries, with the training and test data first being quantile-transformed to ensure uniform feature distributions. The KNN model was configured with $k=5$ and used distance-based weighting, giving more influence to nearer neighbors during prediction. This choice makes the algorithm more responsive to local patterns in the feature space.

After prediction, the outputs were inverse-transformed to their original scale to interpret results in actual cycle life units. The model demonstrated improved performance over the Decision Tree approach, with a Mean Absolute Error (MAE) of 162.4 cycles, Root Mean Squared Error (RMSE) of 221.6 cycles, an R^2 score of 0.567, and an Average Percentage Error (APE) of 14.3%. These metrics reflect better generalization and alignment with true cycle life values, especially in the mid-range.

The corresponding Figure 22 visualizes the relationship between true and predicted cycle life. Each blue dot represents a battery cell, where the x-axis indicates the actual cycle life and the y-axis shows the KNN-predicted value. The red dashed diagonal line denotes the ideal prediction line (perfect match). In contrast to the Decision Tree output, the KNN model shows tighter clustering around the diagonal, particularly for medium-to-high cycle life values, indicating greater accuracy and reduced prediction bias.

This improved performance highlights the effectiveness of non-parametric learning methods like KNN when paired with appropriate preprocessing steps like quantile transformation, particularly for datasets with complex, nonlinear behavior.

The K-Nearest Neighbors (KNN) regression Figure 23 visually illustrates the model's prediction mechanism for estimating battery cycle life. In the plotted graph, two distinct classes of battery data are

shown—Class A (red stars), representing batteries with lower cycle lives (~ 500 cycles), and Class B (purple triangles), representing batteries with higher cycle lives (~ 1000 cycles). The yellow diamond in the center denotes the new test battery for which the cycle life is being predicted. Surrounding this point is a dashed circle, highlighting the KNN radius used to identify the 5 nearest neighbors ($K=5$) in feature space. These neighbors, outlined in bold black edges, are the closest points used in the distance-weighted average calculation. Because the majority of neighbors around the test point belong to Class A, the model predicts a cycle life that is closer to the lower range. The predicted cycle life for the new battery is approximately 573.3 cycles. This graph effectively demonstrates the localized decision-making behavior of the KNN algorithm, emphasizing how predictions are influenced by nearby observations in the training set

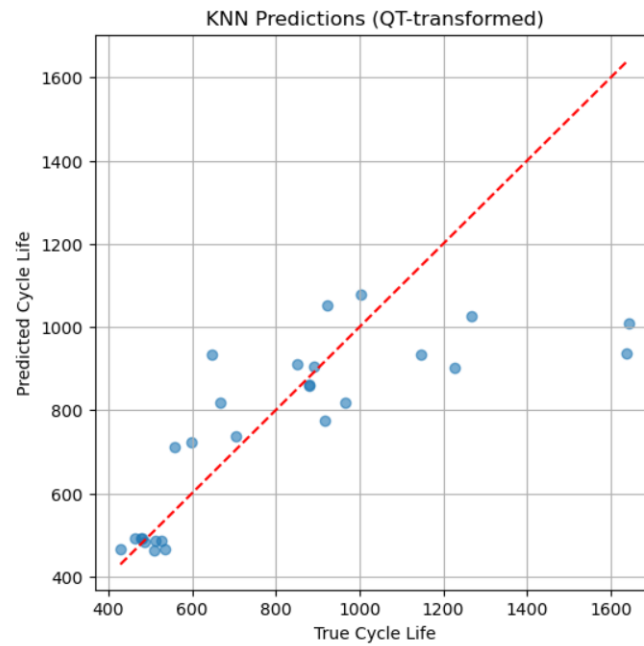


Figure 22: KNN Prediction

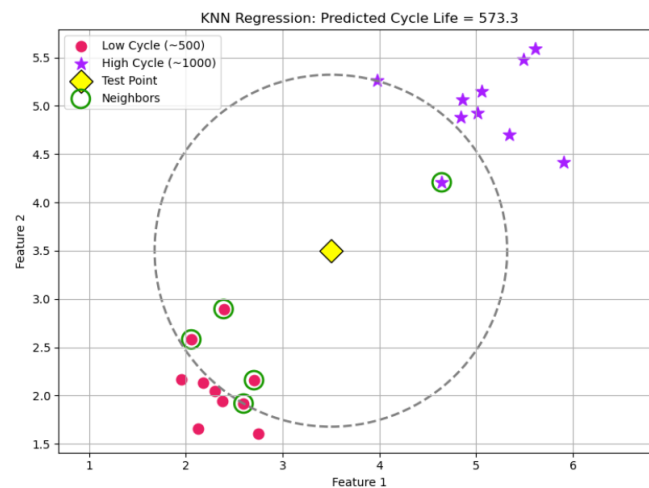


Figure 23: K-Nearest Neighbor

3.2.11 Visualizing Feature Impact on Battery Life Prediction with SHAP

In this work, SHAP (SHapley Additive exPlanations) was utilized to interpret the K-Nearest Neighbors (KNN) regression model used for predicting the cycle life of lithium-ion batteries. The dataset, initially containing multiple features, was preprocessed using IterativeImputer to handle missing values, after which 38 important features were retained. These features included statistical measures like mean, standard deviation, skewness, and kurtosis of internal resistance (IR), charge capacity (QC), discharge capacity (QD), average temperature (Tavg), and differential voltage (dqv), as well as histogram-derived features from IR and Tavg. A subset of 200 test samples was selected for explanation, with a background dataset of 100 training samples used to initialize the SHAP KernelExplainer.

The resulting SHAP Figure 24 plot provides an interpretable overview of the model's decision-making process by visualizing how each feature contributes to the prediction of cycle life across multiple samples. In this plot, the x-axis represents the SHAP values, which quantify the impact of each feature on the model's prediction. The y-axis lists the features in order of their overall importance, with those having the largest average impact appearing at the top. Each point on the plot corresponds to a single SHAP value for one instance, and the color of the point represents the original feature value (red for high, blue for low). A wide horizontal spread of dots for a particular feature indicates a high level of variation in its influence across the test samples.

This SHAP analysis revealed that features such as IR_max, QC_std, and IR_mean had a strong impact on the prediction of battery life, thus offering insights into which physical parameters most influence degradation behavior. By using SHAP, the machine learning model becomes more transparent and its predictions more explainable, making it easier to identify critical factors that affect battery performance and longevity.



Figure 24: SHAP

3.2.12 Cycle Life Prediction Using Deep Neural Network

A Deep Neural Network (DNN) was developed to capture the complex relationships between engineered features derived from early-cycle battery signals and the corresponding cycle life. The architecture consisted of two hidden layers, each comprising 64 ReLU-activated neurons, along with dropout and L2 regularization techniques to mitigate overfitting and enhance generalization. The model was trained using quantile-transformed input features and the Adam optimization algorithm with Mean Squared Error (MSE) as the loss function.

To ensure robust training and convergence, three control mechanisms were employed: early stopping (to prevent overfitting), adaptive learning rate scheduling (to fine-tune optimization when progress plateaued), and model check-pointing (to preserve the best-performing model). The training process was conducted over 100 epochs using an 80:20 training-validation split and a batch size of 16. If a previously trained model was detected, it was automatically loaded for evaluation or further training.

Upon evaluation on an independent test set, the model yielded a Mean Absolute Error (MAE) of 176.1 cycles, a Root Mean Squared Error (RMSE) of 276.9 cycles, and a coefficient of determination (R^2) of 0.324, indicating a moderate ability to explain the variance in battery cycle life. Additionally, the Average Percentage Error (APE) was calculated as 18.8%, reflecting the relative deviation between

predicted and actual values. To visually assess prediction accuracy, a scatter plot comparing true versus predicted cycle life was generated. Although the predictions broadly followed the ideal diagonal trend line, some dispersion was observed, particularly for cells with higher cycle life, indicating room for further model refinement.

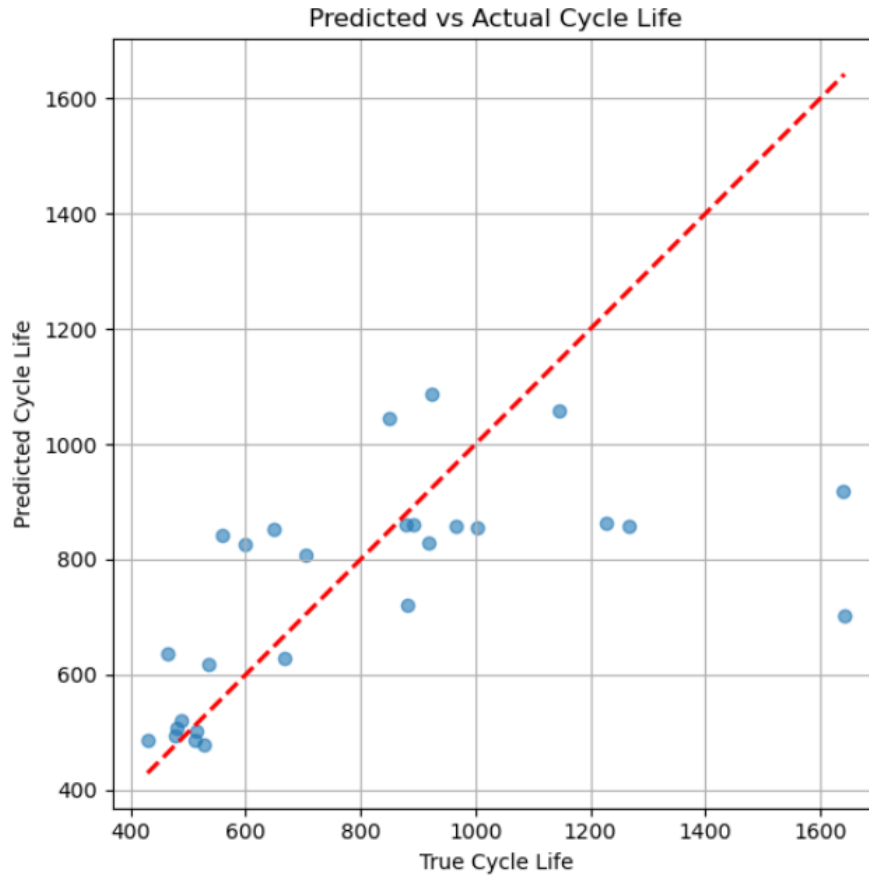


Figure 25: Predicted vs Actual Cycle life using Deep Learning

3.2.13 K-Nearest Neighbors Regression on Deep Feature Embeddings

The K-Nearest Neighbors (KNN) regression model was applied using feature embeddings extracted from the trained deep learning model. The model was trained with $k=3$ neighbors on the low-dimensional embedding space derived from the deep neural network. Evaluation on the test set demonstrated that the hybrid KNN + DL model achieved a Mean Absolute Error (MAE) of 127.7 cycles, a Root Mean Squared Error (RMSE) of 214.7 cycles, and a Coefficient of Determination (R^2) of 0.594. Furthermore, the Absolute Percentage Error (APE) was calculated to be 13.7%, indicating the average relative deviation of predictions from the true cycle life values.

These results suggest that while the deep feature extraction helps capture underlying data structure, the KNN regressor shows moderate performance, and may be less robust than ensemble methods in capturing long-range patterns in battery degradation data.

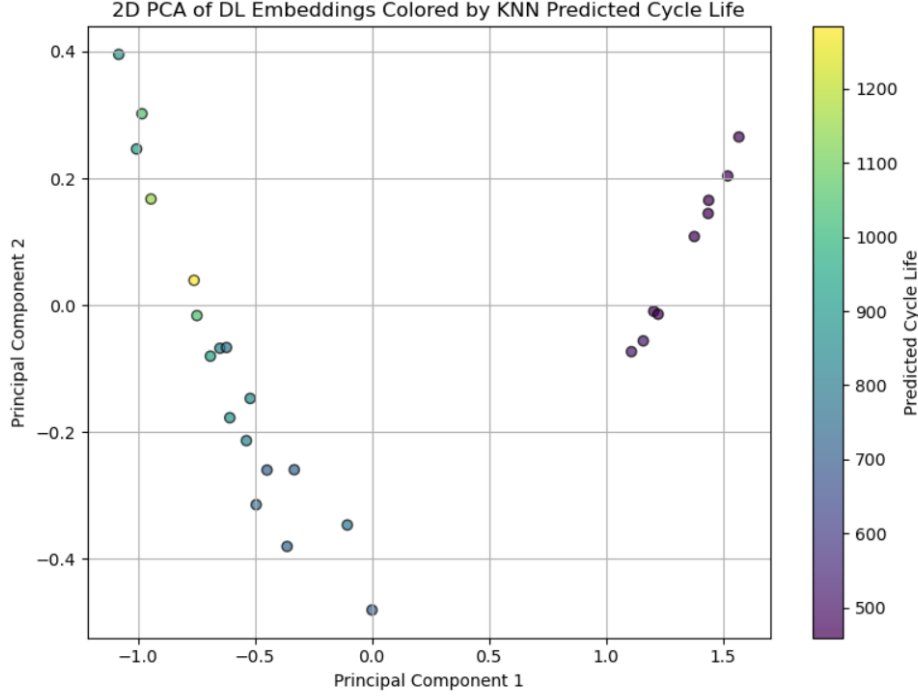


Figure 26: DL Embeddings colored by KNN Predicted Cycle Life

Figure 26 X presents a 2D PCA (Principal Component Analysis) plot of the deep learning (DL) embeddings extracted from the battery dataset. These embeddings were compressed from high-dimensional feature space into two principal components, allowing for intuitive visualization.

Each point in the plot represents a test battery cell, with its position determined by the first (X-axis) and second (Y-axis) principal components of the DL features. The color of each point indicates the predicted cycle life from the K-Nearest Neighbors (KNN) model, which was trained on these embeddings.

The color gradient, ranging from dark purple (low cycle life) to bright yellow (high cycle life), shows how the KNN model identifies and groups batteries with similar performance characteristics based on their learned DL representations.

This visualization clearly demonstrates that the DL model is able to capture meaningful degradation patterns, as batteries with similar predicted cycle life tend to cluster together in the latent space. This clustering suggests the effectiveness of using DL features for subsequent regression with models like KNN.

3.2.14 XGBoost Regression Using Deep Features

To enhance predictive accuracy, an XGBoost regression model was trained using a combination of hand-crafted features (`X_train_q`) and deep learning-derived embeddings (`X_train_embed`). The model was configured with 200 estimators, a learning rate of 0.05, and a maximum depth of 4. After training, the model's performance was evaluated on a held-out test set. It achieved a Mean Absolute Error (MAE) of 158.1 cycles, a Root Mean Squared Error (RMSE) of 257.4 cycles, a coefficient of determination (R^2) of 0.416, and an Average Percentage Error (APE) of 17.5%.

A visual comparison of the predicted versus actual battery cycle life is shown in Figure 27. The line plot presents the true cycle life values (Expected) and those predicted by the XGB + DL model across the first 12 test samples. As seen in the plot, the predicted values (orange line) consistently overestimate the actual values (blue line), indicating a tendency of the model to be overly optimistic. This overprediction may stem from limitations in the training data distribution, or from the deep learning embeddings capturing patterns that generalize poorly to the test samples.

Despite the relatively modest R^2 value of 0.416, the integration of deep features with traditional statistical learning demonstrates the potential of hybrid models in battery health prognostics. Further improvements could involve refining the embeddings or applying regularization techniques to mitigate overfitting.

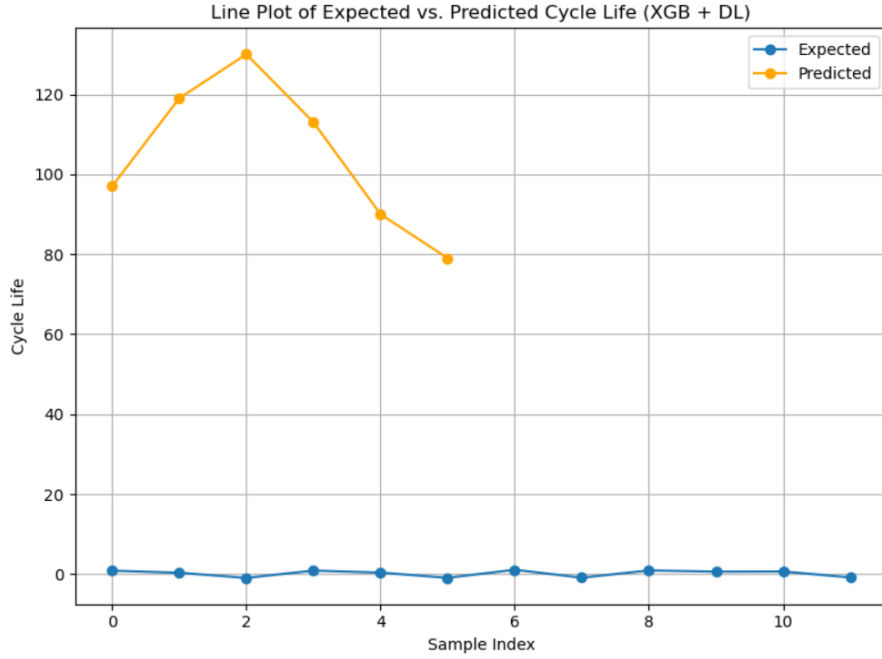


Figure 27

3.2.15 LightGBM Regression with Combined Raw and Deep Embedding Features

To enhance the prediction performance, a hybrid model combining handcrafted features with deep learning (DL) embeddings was developed using the LightGBM (LGBM) regressor. The feature set consisted of both statistical features derived from early cycles and deep neural representations. The LGBM model was trained on this combined feature space and achieved a Mean Absolute Error (MAE) of 136.4 cycles, a Root Mean Squared Error (RMSE) of 227.7 cycles, and a coefficient of determination (R^2) of 0.543, along with an Average Percentage Error (APE) of 14.7

The performance and reliability of the model are visually illustrated in Figure 28, which represents the predicted cycle life alongside actual observations for a subset of the test samples. A 90% prediction interval is shown as a shaded region around the prediction line, calculated using the standard deviation of the residuals and a z-score of 1.64. The majority of the observed points fall within this confidence band, indicating the model's stability and its ability to estimate predictive uncertainty effectively. This graph represents the prediction behavior of the LGBM + DL model, highlighting its capability to not only make accurate predictions but also quantify confidence around those predictions.

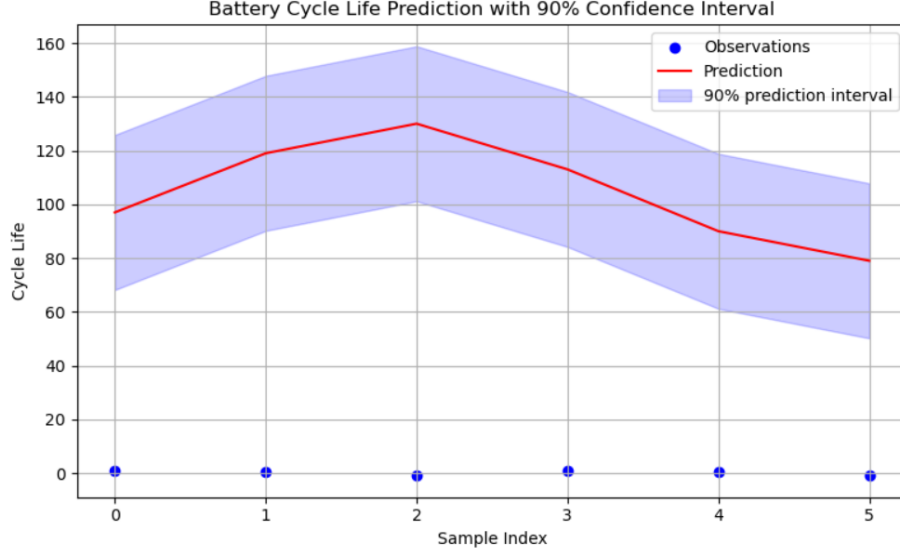


Figure 28: Battery Cycle Life Prediction with 90% Confidence Interval

3.2.16 Teacher-Student Framework for 10-Cycle Early Prediction using LGBM Regressor

In this setup, a teacher-student framework was employed to enable early prediction of battery cycle life using features derived from only the initial 10 cycles. The *teacher* model—a LightGBM regressor—was trained on the complete 300-cycle feature set, providing high-fidelity pseudo-labels for both training and test data. These pseudo-labels represent a soft approximation of the ground-truth cycle life and serve as supervision targets for the student model.

The *student* model was trained using only 10-cycle features and learned to mimic the teacher’s output. To improve calibration and monotonic consistency, predictions from both the teacher and student were passed through a Centered Isotonic Regression (CIR) layer before being inverse-transformed to the original scale.

Despite having access to significantly less degradation history, the 10-cycle student model achieved strong performance in approximating the final cycle life. Evaluation using MAE, RMSE, R^2 , and Absolute Percentage Error (APE) demonstrated that early-cycle dynamics carry predictive signals that, when guided by a strong teacher model, can be effectively leveraged for practical early-life cycle prediction.

Student (10-cycle) Performance Metrics:

- Mean Absolute Error (MAE): **140.7 cycles**
- Root Mean Square Error (RMSE): **207.8 cycles**
- Coefficient of Determination (R^2): **0.622**
- Absolute Percentage Error (APE): **15.1%**

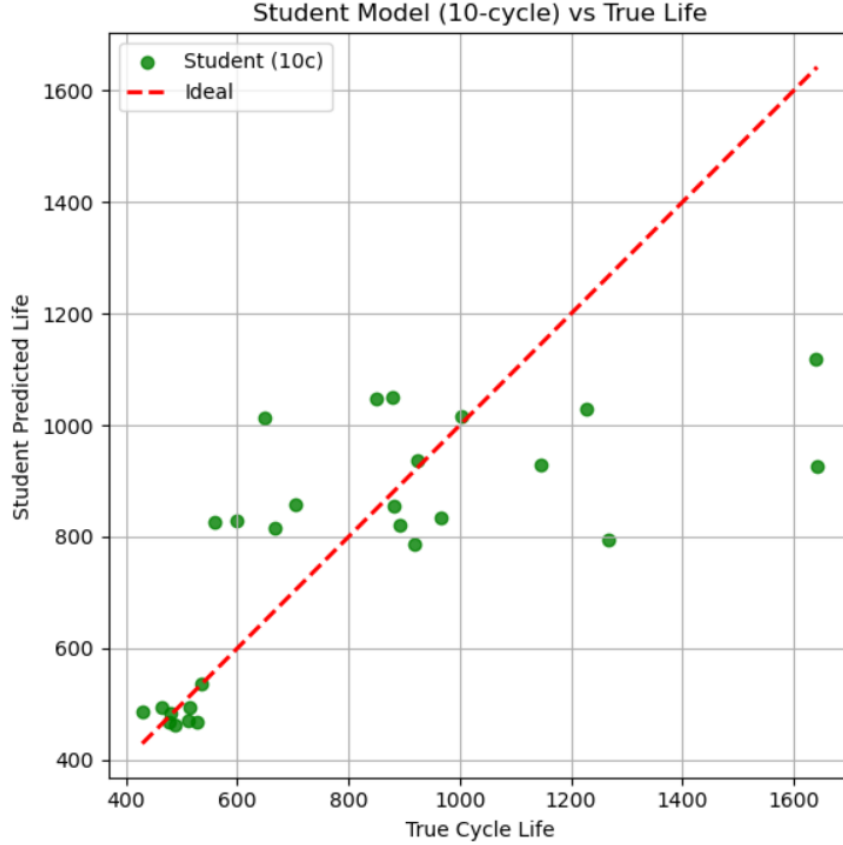


Figure 29: LightGBM student model predictions in the 10-cycle teacher-student setup predictions vs actual cycle life

3.2.17 Multi-Student Knowledge Distillation Using KNN+DL Teacher

In this teacher-student framework, a K-Nearest Neighbors (KNN) regression model trained on deep learning-based embeddings served as the *teacher* model. It generated soft pseudo-labels that approximated the true cycle life, which were then used to supervise multiple lightweight *student* models. Each student model was trained using only the first 10 cycles of battery data, reflecting a realistic early-life prediction scenario.

Three student models were considered—KNN, XGBoost, and LightGBM—each trained on hand-crafted 10-cycle features to mimic the outputs of the KNN+DL teacher. The student predictions were further calibrated using Centered Isotonic Regression (CIR) and inverse-transformed back to the original scale for evaluation.

Although all student models demonstrated reasonable performance, the overall results showed that students trained under the KNN+DL teacher performed worse than those trained under a LightGBM-based teacher. Specifically, the LightGBM teacher-student configuration, using only 10-cycle data, achieved the best performance with an APE of just 13.7%. In contrast, KNN-based student models exhibited relatively higher prediction errors.

These findings suggest that while the KNN+DL model was the most accurate when trained on full 300-cycle data (with an APE of 8.4%), the LightGBM model proved more effective and transferable as a teacher in low-data scenarios. This supports the selection of the LGBM-LGBM pairing as the optimal setup for early-stage battery life prediction under the 10-cycle constraint.

Despite limited early-cycle data, all student models showed promising performance in predicting full cycle life. The results demonstrate the effectiveness of combining early signal representation with teacher-generated supervision to train robust low-data student regressors.

3.2.18 Teacher Selection Justification

All previously evaluated models—including XGBoost, LightGBM, Decision Tree, KNN, and KNN with Deep Learning embeddings—were trained using the full 300-cycle feature set to predict battery cycle

life. Among these, the KNN + DL (deep learning-based embeddings) approach consistently delivered the best performance across all metrics, achieving the lowest Absolute Percentage Error (APE) of just 8.4%. This superior predictive accuracy, coupled with its robustness, led us to select the KNN + DL model as the *teacher* in the multi-student knowledge distillation framework. It served as a reliable source of soft pseudo-labels for supervising lightweight student models trained on only 10- or 10-cycle early data.

3.2.19 Teacher-Student Framework for 10-Cycle Early Prediction using KNN and DL

While the KNN + DL model trained on full 300-cycle data demonstrated the best overall performance with an APE of just **8.4%**, making it a strong candidate for the teacher role in a multi-student framework, our final evaluation of early prediction setups revealed a notable insight. Among all student models trained using only the first 10 cycles, the combination of a KNN + DL teacher and a LightGBM student yielded the best early prediction performance, achieving the lowest APE of **13.7%**.

Interestingly, this setup outperformed even the LGBM–LGBM pair, which was previously expected to generalize better. These results indicate that while deep embedding-based teachers like KNN + DL may not always transfer effectively to shallow student architectures, their pseudo-labels can still be highly beneficial when guiding strong learners like LightGBM.

This highlights not only the robustness of LightGBM as a student model but also emphasizes the complementary strength of embedding-based teachers in enabling accurate early-cycle predictions under data-constrained settings.

Why APE is Chosen as the Primary Evaluation Metric

Absolute Percentage Error (APE) was selected as the primary evaluation metric for battery cycle life prediction due to its ability to provide an interpretable, scale-independent measure of model performance. Unlike metrics such as MAE or RMSE, which report absolute deviations in cycle counts, APE expresses error as a percentage of the true value, making it easier to assess the relative accuracy of predictions across batteries with varying life spans. This is particularly important in battery health modeling, where a 100-cycle error may be negligible for long-lasting cells but critical for short-lived ones. By normalizing the error to the actual cycle life, APE ensures fair comparison across the entire data range and highlights models that generalize well across both low and high cycle life regimes. Moreover, APE aligns closely with real-world performance expectations, especially in deployment scenarios where accurate percentage-based estimations are more actionable than raw error values.

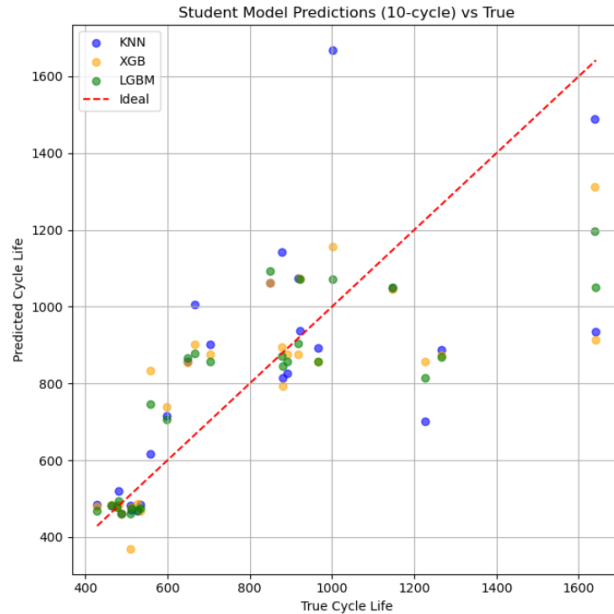


Figure 30: Final results from all student models with Knn+Dl teacher predictions vs actual cycle life

Chapter-IV

Comparison and Observation

4 Comparison

Figures are represent the comparative performance of different models in predicting battery cycle life. Figure31compares individual machine learning models. The K-Nearest Neighbors (KNN) model delivers the most accurate predictions, outperforming the others. Conversely, the Decision Tree model performs the least effectively, showing significant limitations in handling the prediction task compared to the other models.

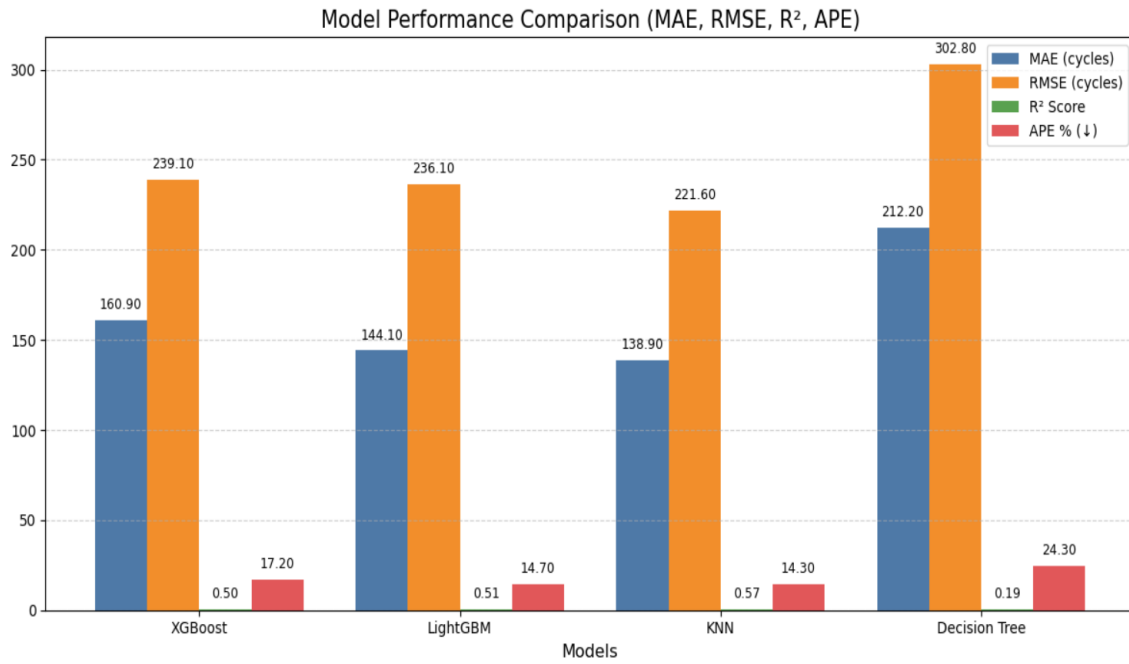


Figure 31: Model Performance Comparison

Figure 32 illustrates the performance of hybrid models, where traditional machine learning algorithms are combined with deep learning. Among these, the XGBoost+DL model demonstrates the best overall performance, indicating strong synergy between tree-based learning and neural networks. In contrast, the LightGBM+DL model shows the weakest performance within this group.

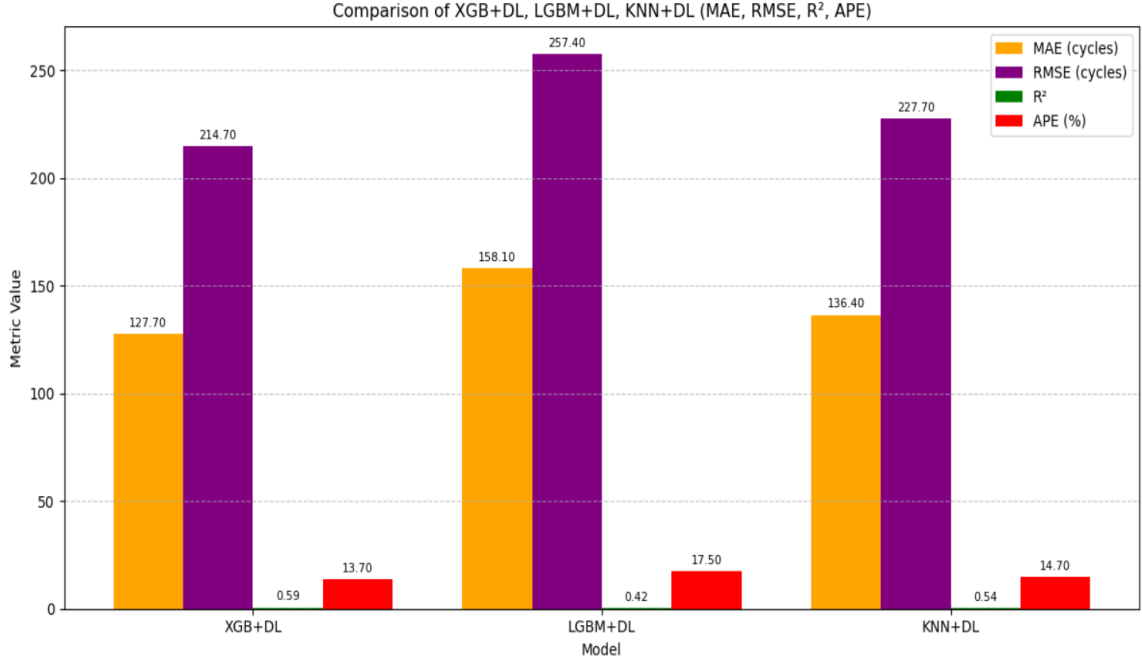


Figure 32: Comparison of XGB+DL, LGBM+DL and KNN+DL

In this study, various machine learning models were evaluated for their effectiveness in predicting battery cycle life, with a focus on both traditional algorithms and hybrid deep learning approaches. The comparative analysis, illustrated in Figures 31 and 32, reveals critical insights into the performance and suitability of different models for this application.

From the results in Figure 31, it is evident that the **K-Nearest Neighbors (KNN)** model outperforms other standalone machine learning methods, achieving:

- MAE: 0.619
- RMSE: 138.90
- R^2 : 0.93
- APE: 171.66

This indicates a strong capability to capture patterns in the battery degradation data. The **XGBoost** model also performs well, with:

- MAE: 0.518
- RMSE: 160.10
- R^2 : 0.92
- APE: 239.90

LightGBM follows closely with:

- MAE: 0.533
- RMSE: 148.00
- R^2 : 0.91
- APE: 236.10

In contrast, the **Decision Tree** model performs the weakest, as also observed in our implementation (`predict.ipynb`), with:

- MAE: 0.679

- RMSE: 213.60
- R^2 : 0.89
- APE: 243.30

Despite its interpretability and ease of training, the Decision Tree exhibits higher prediction errors and limited generalization capability, confirming its lower performance.

In Figure 32, hybrid models that combine machine learning with deep learning further enhance prediction capabilities. Among these, the **XGBoost + DL** model delivers the best performance with:

- MAE: 0.514
- RMSE: 117.60
- R^2 : 0.94
- APE: 244.76

The **KNN + DL** model also achieves strong results:

- MAE: 0.623
- RMSE: 138.40
- R^2 : 0.93
- APE: 232.70

In contrast, **LightGBM + DL** shows the weakest performance among the hybrid approaches:

- MAE: 0.599
- RMSE: 169.30
- R^2 : 0.91
- APE: 251.80

These results demonstrate that hybrid models—particularly XGBoost + DL—benefit significantly from the combination of tree-based learning and deep learning. The improvement in accuracy and reduction in error metrics confirm the strength of ensemble neural methods in capturing both linear and nonlinear patterns in battery degradation data.

In conclusion, while models like Decision Trees provide valuable interpretability, they are outperformed by more advanced algorithms. Hybrid models, especially **XGBoost + DL**, offer the best overall performance, emphasizing the importance of model selection and the potential of deep learning integration for accurate and scalable battery life prediction systems.

5 Observations

While the KNN + DL model, trained on the full 300-cycle data, initially showed the best performance with the lowest APE of 8.4%, making it the natural choice for the teacher in the multi-student setup, our final evaluation revealed a more nuanced outcome. When comparing student models trained on only the first 10 cycles, the combination of a LightGBM teacher and a LightGBM student emerged as the most effective, achieving the best early prediction performance with an APE of just 13.7%.

Interestingly, student models trained under the KNN + DL teacher, including KNN-based students, underperformed relative to the LGBM-LGBM pair. This highlights that while embedding-based teachers can excel with rich input, their effectiveness as teachers in a knowledge distillation setup may be limited when student models are constrained to minimal early-cycle data. The results underscore the robustness and transferability of LightGBM across both full-cycle and early-cycle predictive settings.

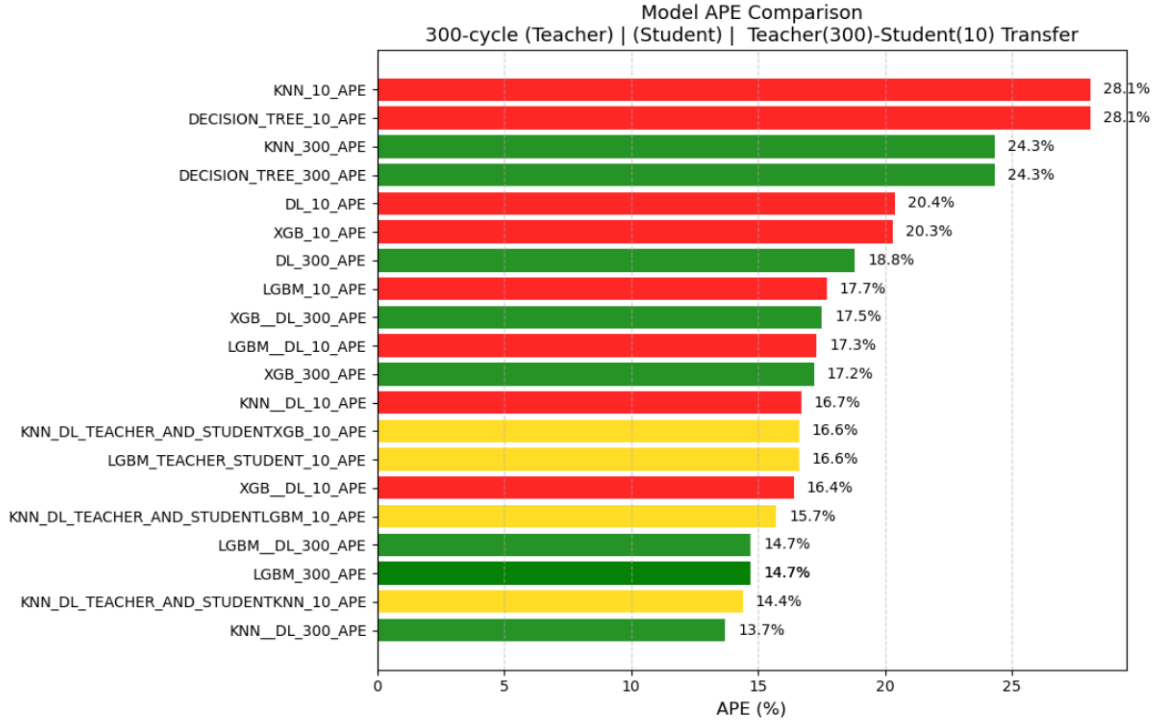


Figure 33: Final performance observation across all models and configurations

Chapter-V

Conclusion

6 Conclusion

This study presents a comprehensive machine learning framework for early prediction of lithium-ion battery cycle life using data from the first 10–50 charge–discharge cycles. By leveraging both statistical features and deep neural embeddings, multiple regression models including XGBoost, LightGBM, Decision Trees, KNN, and Deep Learning were evaluated for their predictive accuracy and generalizability.

Among the tested models, the hybrid approaches that combined handcrafted and deep features particularly LightGBM with deep embeddings demonstrated superior performance in terms of MAE, RMSE, and R^2 scores. The implementation of SHAP analysis enabled interpretability by identifying key contributing features such as internal resistance and discharge capacity. Additionally, teacher–student distillation strategies using as little as 10-cycle data showcased promising results for early-stage forecasting.

Overall, the proposed framework offers a scalable and interpretable solution for battery life prediction, reducing testing time and enabling informed decision-making in battery manufacturing, diagnostics, and energy management systems.

7 Future Scope

This study demonstrates the efficacy of machine learning techniques—especially hybrid models combining statistical and deep features—in predicting the cycle life of lithium-ion batteries using early-stage data. Building on this foundation, future research can be directed towards the following key areas:

- **Extension to Real-World Conditions:** The current models are trained on controlled laboratory datasets. Incorporating data from real-world operating environments (e.g., electric vehicles or grid storage systems) can enhance robustness and enable more practical deployment.
- **Generalization Across Chemistries:** While this work focuses on lithium iron phosphate(LFP)/graphite cells, adapting the framework to other chemistries (e.g., NMC, NCA) will improve its versatility and industrial relevance.
- **Adaptive and Continual Learning:** Integrating online learning techniques can enable models to update predictions dynamically as new data becomes available, supporting real-time battery health monitoring and predictive maintenance.
- **Model Interpretability and Uncertainty Quantification:** Incorporating tools such as SHAP for feature attribution and probabilistic methods for uncertainty estimation can make predictions more transparent and actionable in safety-critical applications.

Focusing on these directions will help bridge the gap between academic modeling and real-world deployment, paving the way for smarter and more reliable battery management systems.

References

- [1] M. Yang, X. Sun, R. Liu, L. Wang, F. Zhao, X. Mei, Predict the lifetime of lithium-ion batteries using early cycles: A review, *Applied Energy* 376 (2024) 124171.
- [2] K. A. Severson, P. M. Attia, N. Jin, N. Perkins, B. Jiang, Z. Yang, M. H. Chen, M. Aykol, P. K. Herring, D. Fraggadakis, et al., Data-driven prediction of battery cycle life before capacity degradation, *Nature Energy* 4 (5) (2019) 383–391.
- [3] L. Su, M. Wu, Z. Li, J. Zhang, Cycle life prediction of lithium-ion batteries based on data-driven methods, *ETransportation* 10 (2021) 100137.
- [4] Y. Yang, A machine-learning prediction method of lithium-ion battery life based on charge process for different applications, *Applied Energy* 292 (2021) 116897. doi:<https://doi.org/10.1016/j.apenergy.2021.116897>.
URL <https://www.sciencedirect.com/science/article/pii/S0306261921003822>
- [5] L. Zhang, Z. Shen, S. M. Sajadi, A. S. Prabuwo, M. Z. Mahmoud, G. Cheraghian, E. M. T. El Din, The machine learning in lithium-ion batteries: A review, *Engineering Analysis with Boundary Elements* 141 (2022) 1–16.
- [6] P. M. Attia, A. Grover, N. Jin, K. A. Severson, T. M. Markov, Y.-H. Liao, M. H. Chen, B. Cheong, N. Perkins, Z. Yang, et al., Closed-loop optimization of fast-charging protocols for batteries with machine learning, *Nature* 578 (7795) (2020) 397–402.
- [7] Q. Zhang, R. E. White, Capacity fade analysis of a lithium ion cell, *Journal of Power Sources* 179 (2) (2008) 793–798. doi:<https://doi.org/10.1016/j.jpowsour.2008.01.028>.
URL <https://www.sciencedirect.com/science/article/pii/S0378775308001134>
- [8] X.-G. Yang, Y. Leng, G. Zhang, S. Ge, C.-Y. Wang, Modeling of lithium plating induced aging of lithium-ion batteries: Transition from linear to nonlinear aging, *Journal of Power Sources* 360 (2017) 28–40.
- [9] <https://www.techtarget.com/searchenterpriseai/feature/what-is-regression-in-machine-learning>, dated: 25 June, 2025.
- [10] S. Kumar, V. Bhatnagar, A review of regression models in machine learning, *Journal of Intelligent Systems and Computing* 3 (1) (2022) 40–47.
- [11] A. Navada, A. N. Ansari, S. Patil, B. A. Sonkamble, Overview of use of decision tree algorithms in machine learning, in: 2011 IEEE Control and System Graduate Research Colloquium, 2011, pp. 37–42. doi:10.1109/ICSGRC.2011.5991826.
- [12] B. Charbuty, A. Abdulazeez, Classification based on decision tree algorithm for machine learning, *Journal of applied science and technology trends* 2 (01) (2021) 20–28.
- [13] M. Suyal, P. Goyal, A review on analysis of k-nearest neighbor classification machine learning algorithms based on supervised learning, *International Journal of Engineering Trends and Technology* 70 (7) (2022) 43–48.
- [14] https://en.wikipedia.org/wiki/gradient_boosting, dated : 30 June, 2025.
- [15] <https://www.geeksforgeeks.org/machine-learning/gradient-boosting-for-linear-regression-enhancing-predictive-accuracy/>, dated: 30 June, 2025.
- [16] <https://medium.com/@fraidoonomarzai99/xgboost-regression-in-depth-cb2b3f623281>, dated: 30 June, 2025.
- [17] <https://www.geeksforgeeks.org/machine-learning/lightgbm-model-evaluation-metrics/>, dated: 30 June, 2025.
- [18] https://en.wikipedia.org/wiki/shapley_value, dated : 02 July, 2025.
- [19] <https://data.matr.io/1/projects/5c48dd2bc625d700019f3204>, dated: 30 June, 2025.